Week 5: Introduction to Web Applications

RENE OLUOCH

CS-CNS09-25030

Contents

Introduction 2
Introduction to Web Applications 2
Web Application Layout 3
Front End vs. Back End 5
HTML 6
Cascading Style Sheets (CSS) 8
JavaScript 10
Sensitive Data Exposure 11
HTML Injection 14
Cross-Site Scripting (XSS) 16
Cross-Site Request Forgery (CSRF) 18
Back End Servers 19
Web Servers 21
Databases 23
Development Frameworks & APIs 25
Common Web Vulnerabilities 27
Public Vulnerabilities 29
Next steps 32
Conclusion 32

Introduction

In the contemporary digital landscape, web applications have become integral to the functioning of modern enterprises and everyday life. They serve as the backbone of countless services, including e-commerce platforms, online banking, collaborative tools, and content delivery systems. Their accessibility and scalability have contributed to their widespread adoption across various industries. However, with increased functionality comes an expanded attack surface, making web application security a paramount concern. This comprehensive study of web applications has provided a foundational understanding of their architecture, components, development methodologies, and the numerous security challenges they present. Through this exploration, the significance of adopting a secure-by-design approach in web application development and maintenance has become abundantly clear.

Introduction to Web Applications

In this foundational exploration of web applications, I gained a comprehensive understanding of how these systems operate, their architecture, and their security implications. Web applications are dynamic, interactive programs that run within web browsers, structured with client-side components (the front end) and server-side elements (the back end). These applications differ significantly from static websites by responding to user interactions and delivering personalized experiences. Unlike traditional Web 1.0 sites that displayed fixed content, Web 2.0 applications allow real-time changes and functionality. As a result, platforms like Gmail, Amazon, and Google Docs represent modern web applications, offering interactive experiences that adjust dynamically based on user input.

I also learned how web applications differ from native OS applications. Unlike native software that must be installed and is platform-specific, web applications are universally accessible via browsers and do not consume local storage. They benefit from centralized version control, which allows developers to roll out updates globally without user intervention. However, native applications maintain an edge in performance and access to system-level capabilities. Recent advancements in hybrid and progressive web applications now combine the best of both worlds by leveraging native OS resources while maintaining cross-platform accessibility.

Additionally, I explored the various means of web application distribution. Open-source solutions like WordPress and Joomla allow for extensive customization and self-hosting, while proprietary platforms such as Shopify and Wix offer managed services with subscription models. These platforms power millions of applications across the internet, serving diverse business and user needs.

A critical part of this learning journey involved understanding the security challenges web applications face. Due to their global accessibility and complex architectures, web applications present vast attack surfaces. They are frequent targets for malicious actors who exploit vulnerabilities like SQL injection, insecure file uploads, IDOR, and broken access control. Successful attacks can compromise sensitive data, interrupt business operations, or lead to unauthorized system access. I came to appreciate the importance of web application penetration testing in identifying and mitigating these threats. Regular testing, secure development practices, and adherence to standards like the OWASP Web Security Testing Guide are essential for defending these assets.

From an offensive security perspective, the dynamic nature of web applications makes them particularly appealing for attackers. Even small code changes can introduce critical vulnerabilities. For example, manipulating a registration form parameter like roleid might grant unintended admin privileges. Similarly, SQL injection vulnerabilities can be chained with password spraying attacks against Active Directory-backed portals to breach deeper into an organization's network. Such examples underscore the cascading impact a single flaw can have across infrastructure.

Ultimately, understanding web applications is vital for any penetration tester. These systems are ubiquitous, and real-world engagements often involve complex web apps where finding and exploiting subtle flaws can differentiate an average tester from an expert. Mastering the front-end languages (HTML, CSS, JavaScript), understanding client-server interactions, and assessing both authenticated and unauthenticated states are all crucial skills. As I continue this journey, I recognize the value in becoming deeply familiar with web technologies and their associated security implications. This knowledge will allow me to effectively identify and exploit web vulnerabilities while developing a security-first mindset for both offense and defense in modern environments.

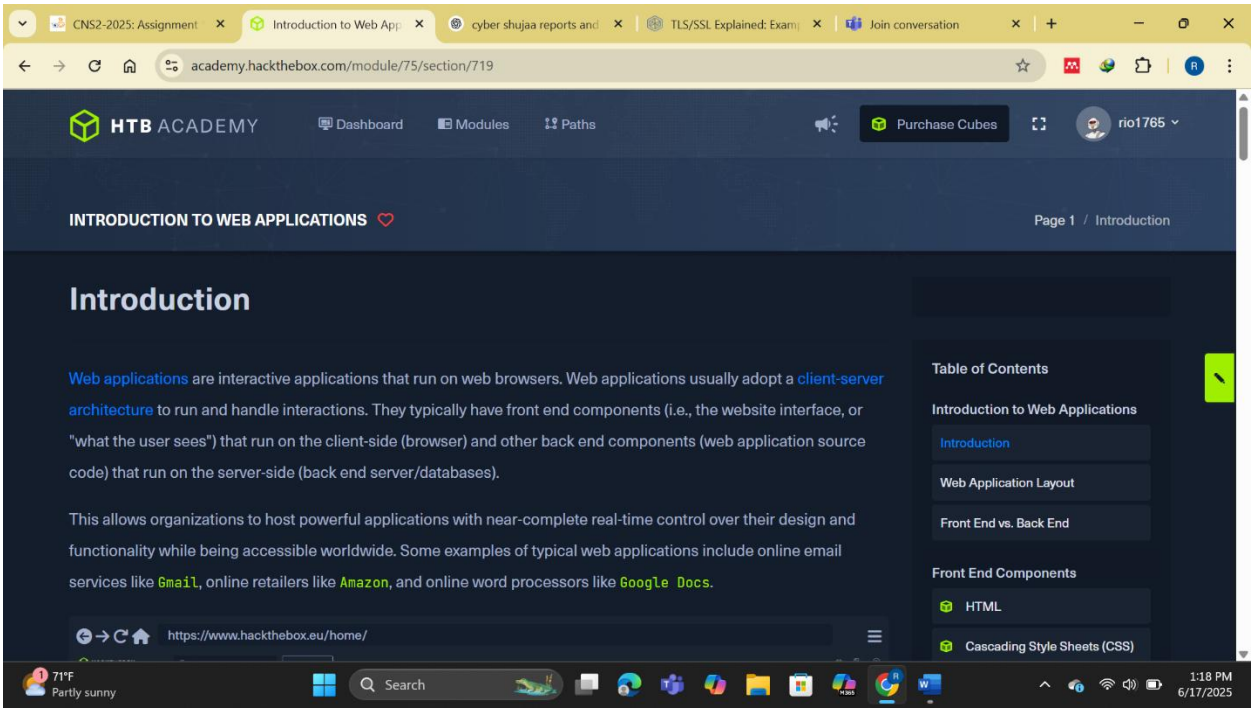

Fig 1.0 introduction

Web Application Layout

In this detailed overview of web application layout, I expanded my understanding of how web applications are structured, deployed, and secured. I learned that web applications can have widely varying designs depending on their business goals and audience, but generally, they follow a three-layer model composed of infrastructure, components, and architecture. The infrastructure encompasses how and where the application is hosted—including where the servers and databases reside—while the components involve the various building blocks like clients, servers, databases, and microservices. Finally, the architecture defines the relationship and interactions between these components.

The infrastructure can follow several models, starting with the simple "One Server" setup where everything runs on a single machine—an easier but riskier approach due to the single point of failure. A more resilient approach is separating the web server and database (Many Servers - One Database), allowing for better segmentation and reduced attack surface. The most robust setup is the Many Servers - Many Databases model, where each application uses its own isolated database. This design supports redundancy and fault tolerance and is considered more secure, albeit more complex to implement. I also learned about modern infrastructure paradigms like microservices and serverless architecture, which support flexibility, scalability, and more efficient deployment. Microservices allow applications to be broken into smaller, independently deployable components, each responsible for a specific task like user registration or payment processing. These services communicate statelessly and can be written in different languages, encouraging agile development. Serverless, offered by platforms like AWS and Azure, removes the need to manage infrastructure, allowing developers to deploy code in containers without managing servers directly.

On the component level, a web application is generally made up of three key elements: the client (user interface in the browser), the server (which processes requests), and the database (where data is stored and retrieved). More advanced applications may also include services like microservices, third-party integrations, and serverless functions. These components interact in a structure often referred to as the Three-Tier Architecture, with the presentation layer (HTML, CSS, JS), application layer (logic, validation), and data layer (database queries and storage) working together to deliver functionality.

From a security standpoint, I learned how critical architectural decisions are. A flaw may not lie in the application code but rather in how the application was designed. For example, poor implementation of access controls like RBAC may expose admin functionalities to regular users. Even if code is secure, design flaws in segmentation or privilege enforcement could present critical vulnerabilities. It’s possible that even after exploiting the back end, I might not find the database, indicating a separate server with its own access controls. This highlights the need for security consideration from the planning phase and continuous testing throughout the development lifecycle.

In essence, understanding the layout of web applications isn't just about knowing how they are built but also why they are built that way and where their weaknesses might lie. As I continue honing my web application security testing skills, this architectural knowledge will be instrumental in helping me map the attack surface, understand back-end logic, identify trust boundaries, and assess what damage might result from exploiting a given vulnerability.

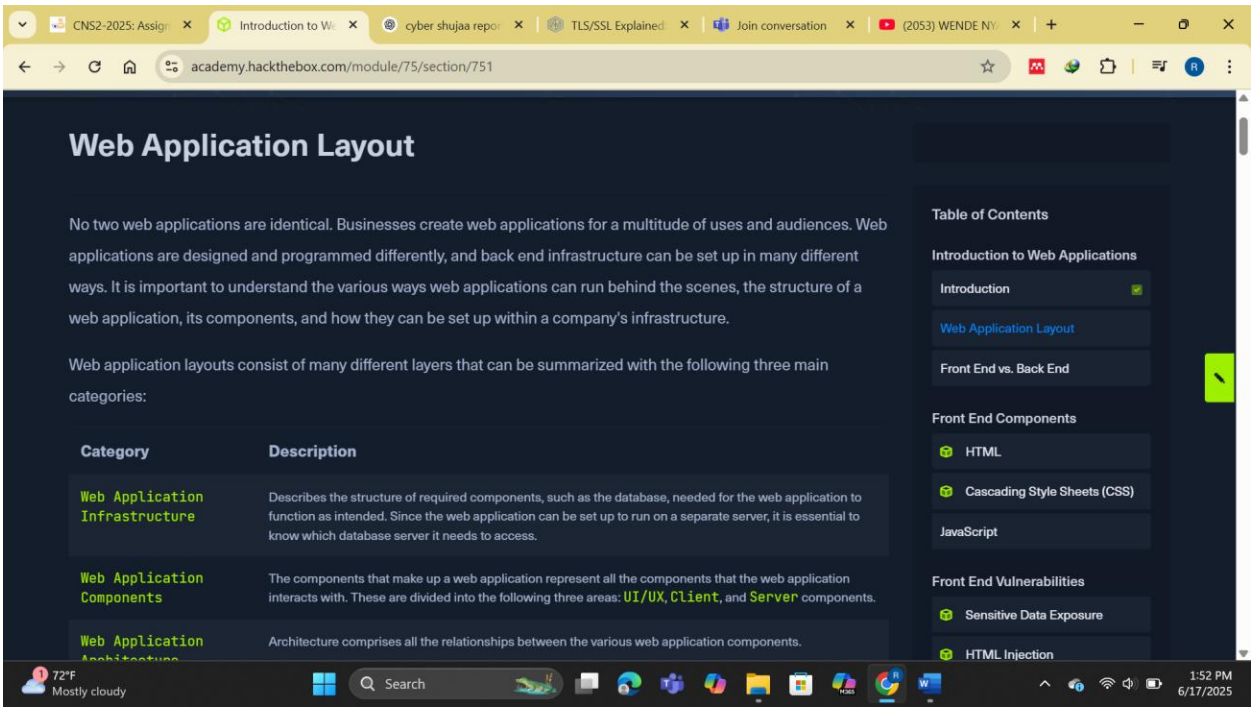

Fig 1.1 web application layout

Front End vs. Back End

In this module, I deepened my understanding of the core difference between front end and back end development, which together form the foundation of any web application. The front end represents the client-side, meaning everything the user directly sees and interacts with through their browser. It includes HTML for structure, CSS for styling and design, and JavaScript for dynamic behavior. The front end is responsible for rendering the user interface, managing responsiveness across different devices and screen sizes, and delivering a fluid user experience (UI/UX). I also learned how poor front end optimization can significantly impact performance, even giving the false impression that the issue lies with the server or internet connection.

On the other hand, the back end operates server-side and handles the core application logic and data processing that the user doesn't see. It includes the web server (like Apache or NGINX), the database (such as MySQL, MSSQL, or MongoDB), the development framework (like Django or Laravel), and the operating system that hosts them all. Back end components process user requests, interact with databases, serve application logic, and integrate with other services. These components can be isolated in different containers or servers to improve security through segmentation—meaning if one part is compromised, the others remain protected.

Security is critical on both ends. While front end vulnerabilities like Cross-Site Scripting (XSS) can lead to client-side attacks, back end vulnerabilities can result in severe outcomes such as unauthorized database access or remote code execution. For instance, if a web application fails to sanitize user inputs properly, it becomes vulnerable to SQL Injection or Command Injection, allowing an attacker to extract or manipulate sensitive data or even take over the system. As testers, we perform whitebox testing when front end source code is accessible, while in most real-world scenarios, back end testing is often blackbox

unless the application is open source or we exploit a vulnerability (like Local File Inclusion) to retrieve the code.

I also reviewed a list of 20 common developer mistakes—like storing passwords in plain text, relying too heavily on the client-side, or failing to encrypt data—that often lead to the OWASP Top 10 vulnerabilities. These include familiar categories such as Broken Access Control, Injection, and Security Misconfiguration. Understanding these flaws prepares me to not only find and exploit them but also to explain them clearly to clients in real-world engagements.

Altogether, this knowledge underscores the importance of approaching web application testing with a full-stack perspective—being able to analyze both what the user sees and what happens behind the scenes. It's essential to identify potential risks in both the visual and hidden components of an application, assess how they interact, and determine how an attacker might use one layer to compromise another. This dual-layered insight is foundational for competent and comprehensive penetration testing.

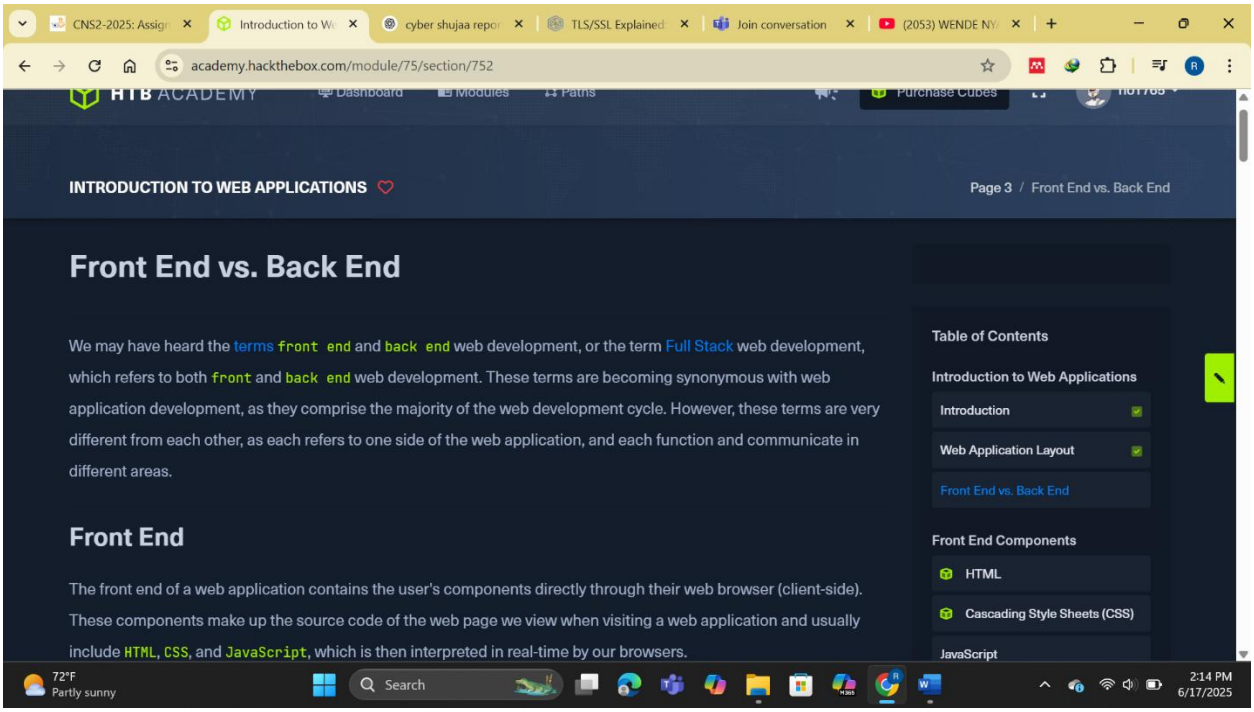

Fig 1.2 front vs back end

HTML

In this section, I focused on understanding the essential role of **HTML (HyperText Markup Language)** as the foundational component of front end web development. HTML structures the content of web pages and defines the page's elements—such as headings, paragraphs, forms, and images—that are rendered by a user's web browser. I examined a simple HTML document example that includes a <head> section for metadata like the page title and a <body> section where visible elements like <h1> headings and <p> paragraphs are defined. These elements follow a nested, tree-like hierarchy that mirrors the **Document Object Model (DOM)**—a structured representation of the page which browsers and scripts use to interact with and manipulate the content dynamically.

Each HTML element is wrapped in an opening and closing tag that determines its type and role. Additionally, HTML elements can include attributes such as id and class to allow for styling via CSS or manipulation through JavaScript. Understanding this structure is crucial for both development and security testing, as it enables pinpointing exactly where elements exist within the page and how they can be targeted—either for intended behavior or in the case of testing for vulnerabilities like **Cross-Site Scripting (XSS)**.

I also learned about **URL encoding**, which is essential for ensuring that special characters can be safely transmitted in URLs. Since URLs can only include a subset of ASCII characters, other characters must be percent-encoded. For example, a space character becomes %20, and a single quote (') becomes %27. This encoding helps browsers correctly interpret characters that would otherwise be misread or blocked. Tools like Burp Suite and various online utilities can easily convert between encoded and decoded forms, which is helpful when testing or crafting payloads in security assessments.

Furthermore, I gained insight into how different DOM levels—such as document.head, document.body, or document.h1—can be navigated and interacted with programmatically. This knowledge is particularly valuable in offensive security, where manipulating the DOM structure or inserting elements via JavaScript is a common technique during front-end exploitation.

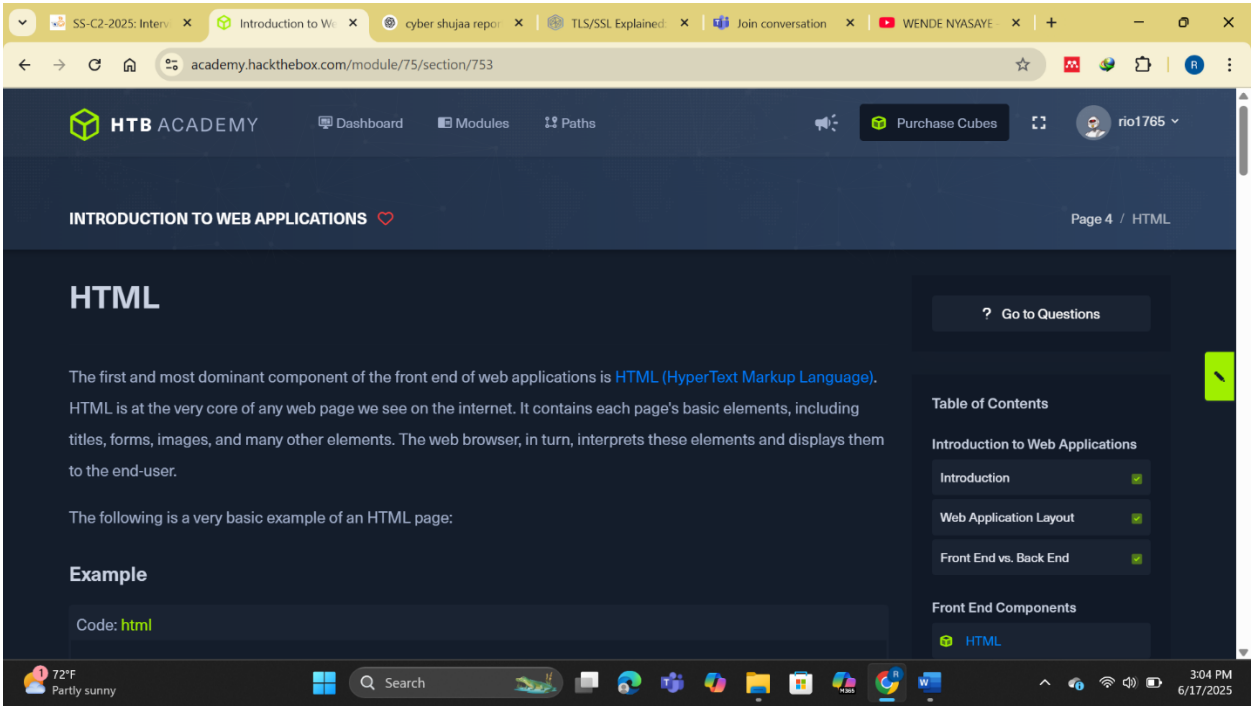

Fig 1.3 html

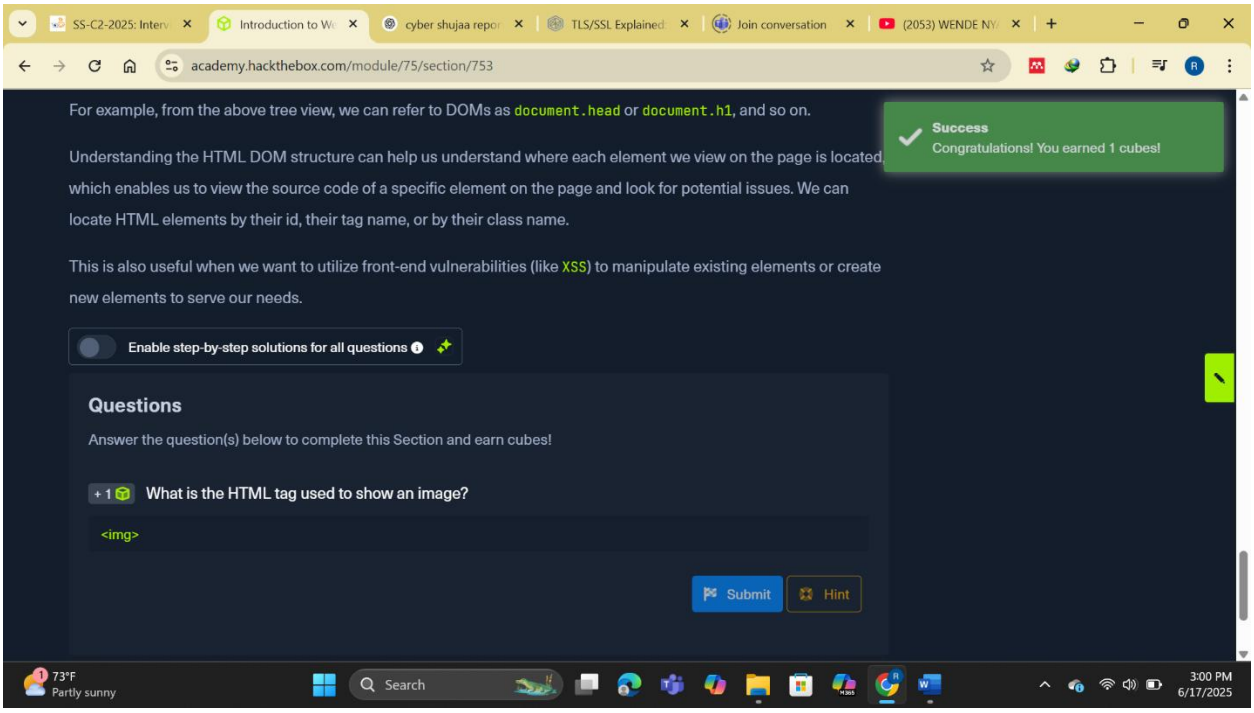

Fig 1.4 answer to html

Cascading Style Sheets (CSS)

In this section, I deepened my understanding of **Cascading Style Sheets (CSS)**—the language responsible for styling and formatting HTML elements within web applications. CSS works hand-in-hand with HTML to control the presentation layer of a web page, defining how content appears to the user. While HTML builds the structural skeleton of a web page, CSS gives it visual life through properties like **colors, fonts, layout alignment, positioning, spacing, and even animations**.

I explored how CSS syntax is structured around **selectors** (e.g., body, h1, or .className) and **property-value pairs** enclosed in curly braces. For example, h1 { color: white; text-align: center; } applies white color and center-alignment to all <h1> headers on a page. This syntax allows developers to customize any HTML element by either targeting its tag name, class, or unique ID. This modular approach makes it easy to maintain and change the design of an entire site by updating just the CSS rules.

Beyond basic formatting, I learned that CSS can power advanced **visual effects and animations**. Using properties like @keyframes, animation-duration, and animation-direction, CSS can create transitions and dynamic behavior, enhancing user interaction. CSS is also frequently paired with JavaScript to make **dynamic, responsive pages** that adapt based on user input, such as keystrokes or mouse movement. An example highlighted in the material was the "Parallax Depth Cards" on CodePen, which combine CSS, HTML, and JavaScript to create stunning motion and depth effects.

In addition to writing raw CSS, developers often turn to **CSS frameworks** that simplify and speed up design workflows. Frameworks like **Bootstrap, SASS, Foundation, Bulma, and Pure** offer pre-built,

reusable styles for common interface components—buttons, forms, navigation bars, etc.—which are especially useful in building responsive, mobile-friendly applications. These frameworks also integrate well with JavaScript and are designed to accommodate modern web app needs.

Overall, CSS is not just a tool for making things look nice—it is a **powerful language that shapes user experience**, accessibility, and even brand identity. Whether applied manually or via frameworks, CSS is foundational in front-end development and an essential skill for anyone looking to understand or test web applications.

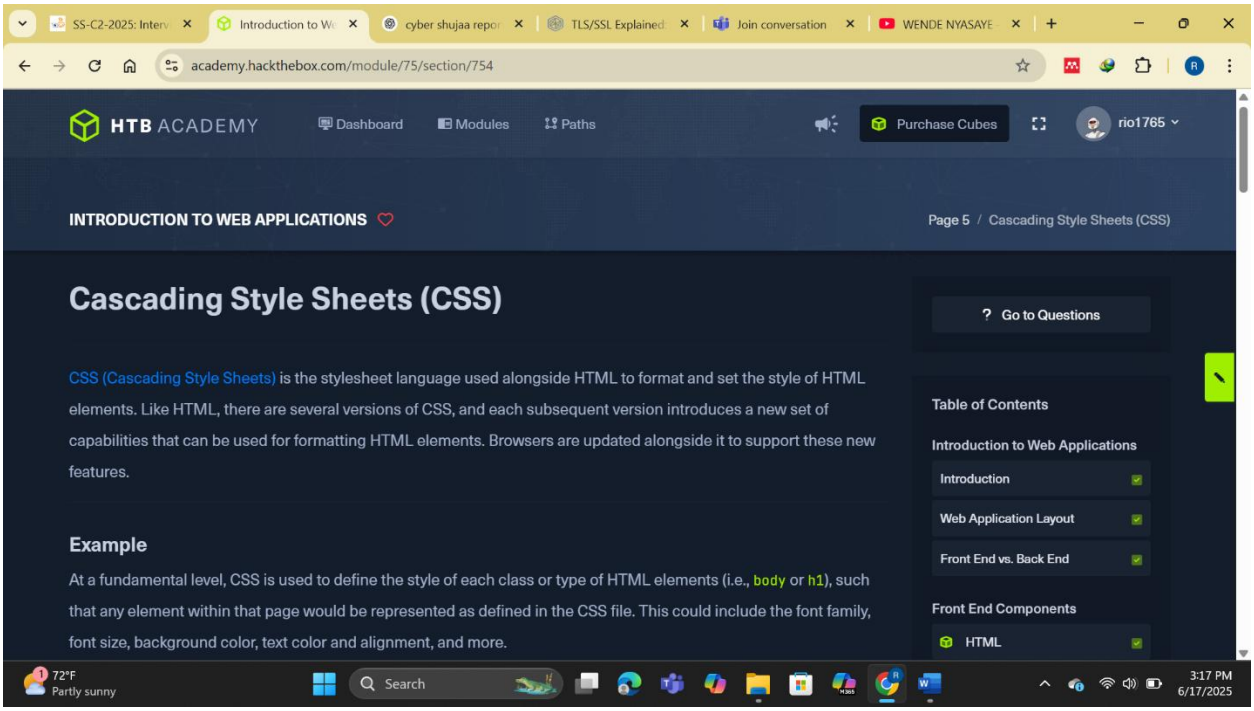

Fig 1.5 css

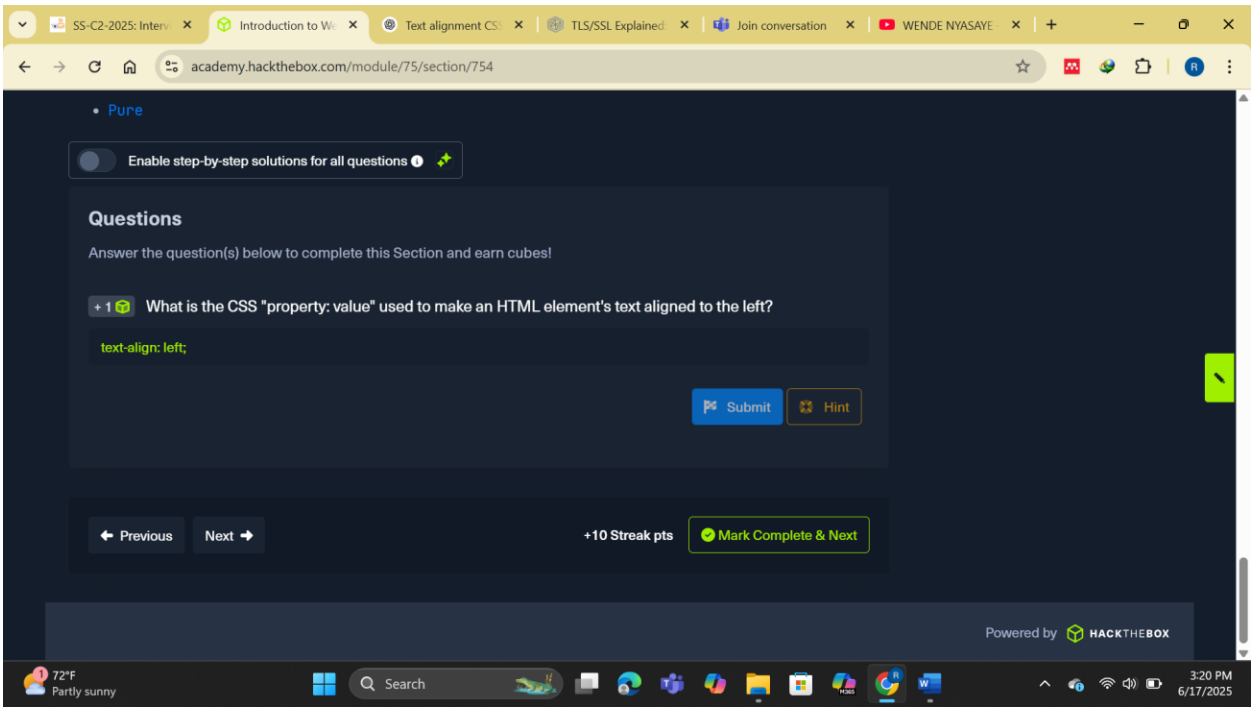

Fig 1.6 answer to question on css

JavaScript

JavaScript is an essential and dynamic scripting language that powers much of the interactivity we experience on modern websites. While HTML and CSS are primarily responsible for a web page's structure and style, **JavaScript adds life to a page by enabling real-time, client-side functionality and user interactivity**. It is one of the most widely used programming languages globally and serves as the backbone of modern web development. JavaScript code is embedded into web pages using the <script> tag, and it can either be written inline within the page's source or referenced externally using a src attribute pointing to a JavaScript file.

For example, a basic JavaScript function might use document.getElementById("button1").innerHTML = "Changed Text!"; to dynamically alter the content of a web page element, such as updating a button's label when it's clicked. This ability to manipulate the Document Object Model (DOM) allows developers to create highly responsive web pages that react to user input without requiring a page reload. This interactivity is further enhanced by technologies like **Ajax**, which allows JavaScript to send or receive data from the back end without disrupting the user's browsing experience.

JavaScript plays a critical role not just in interface updates and form handling, but also in more complex operations like validating user input, dynamically generating HTML content, making asynchronous API requests, and creating advanced animations in combination with CSS. The performance benefit of JavaScript stems from the fact that all modern browsers come with built-in JavaScript engines (like Chrome's V8 engine), allowing scripts to be executed directly on the client's machine without relying on server-side processing, thereby reducing load times and improving responsiveness.

Due to its vast scope and complexity, modern JavaScript development has been greatly streamlined by the introduction of **JavaScript frameworks** and **libraries**, which simplify common programming tasks and enable developers to build large-scale web applications more efficiently. Libraries like **jQuery** simplify DOM manipulation and event handling, while full-fledged frameworks like **Angular**, **React**, and **Vue** provide structured approaches to building scalable, component-based front-end applications. These tools support features such as real-time data binding, routing, templating, and state management, making them indispensable in the development of modern web apps.

In summary, JavaScript transforms a static webpage into a **dynamic, interactive experience**. It empowers developers to build rich user interfaces, manage data asynchronously, and improve client-side responsiveness. Combined with frameworks and development tools, JavaScript is a critical pillar of modern web application development, ensuring both functionality and performance across devices and platforms.

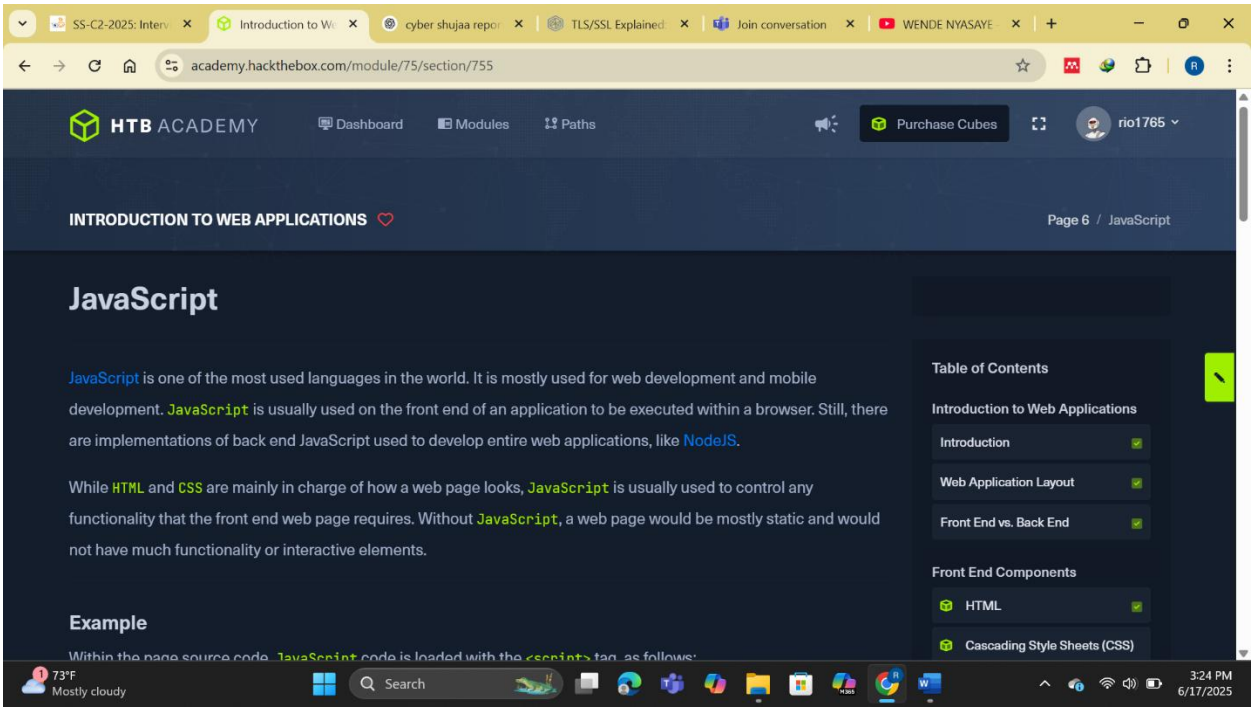

Fig 1.7 java script

Sensitive Data Exposure

As I explored the security aspects of front-end web development, I learned how crucial it is to analyze client-side components for potential vulnerabilities, even though they execute in the browser and not directly on the server. While these components—built using HTML, CSS, and JavaScript—may not compromise the backend infrastructure on their own, I discovered that they can still put users at serious risk if misconfigured or poorly coded. One of the most important takeaways for me was understanding how **Sensitive Data Exposure** can occur through something as simple as developer comments or unfiltered information in the page source.

For example, I learned to always begin testing a web application by viewing its page source, which can be done by right-clicking anywhere on the page and selecting “View Page Source,” or using Ctrl+U as a shortcut. This exposes the HTML, JavaScript, and links to external resources. While inspecting one such page, I found a comment that said <!-- TODO: remove test credentials test:test -->. This was a reminder left by a developer, but it had not been removed before going live—exposing what could be valid login credentials to anyone with basic knowledge of browser tools.

This experience showed me that even seemingly harmless things like HTML comments or test links can provide valuable information to an attacker. I realized how exposed login credentials, hidden directories, or even API keys could easily be left behind in production by accident. It was also clear to me that such information could be used as a foothold for deeper exploitation, like accessing admin panels or manipulating the server’s behavior.

To prevent these issues, I learned that front-end code should be carefully reviewed and cleaned before deployment. Unused or debug code should be removed, and sensitive logic should never rely on client-side enforcement. I also discovered that developers often use **JavaScript obfuscation** or **minification** techniques to make it harder for attackers to reverse-engineer their code, and that automated tools like Burp Suite can help uncover vulnerabilities more efficiently.

This section helped me appreciate the importance of not overlooking front-end components during a penetration test. It reminded me that even if the back-end appears secure, a poorly secured front end can act as an entry point. I now understand that reviewing front-end code isn’t just about aesthetics or functionality—it’s a fundamental step in identifying and mitigating real-world security threats.

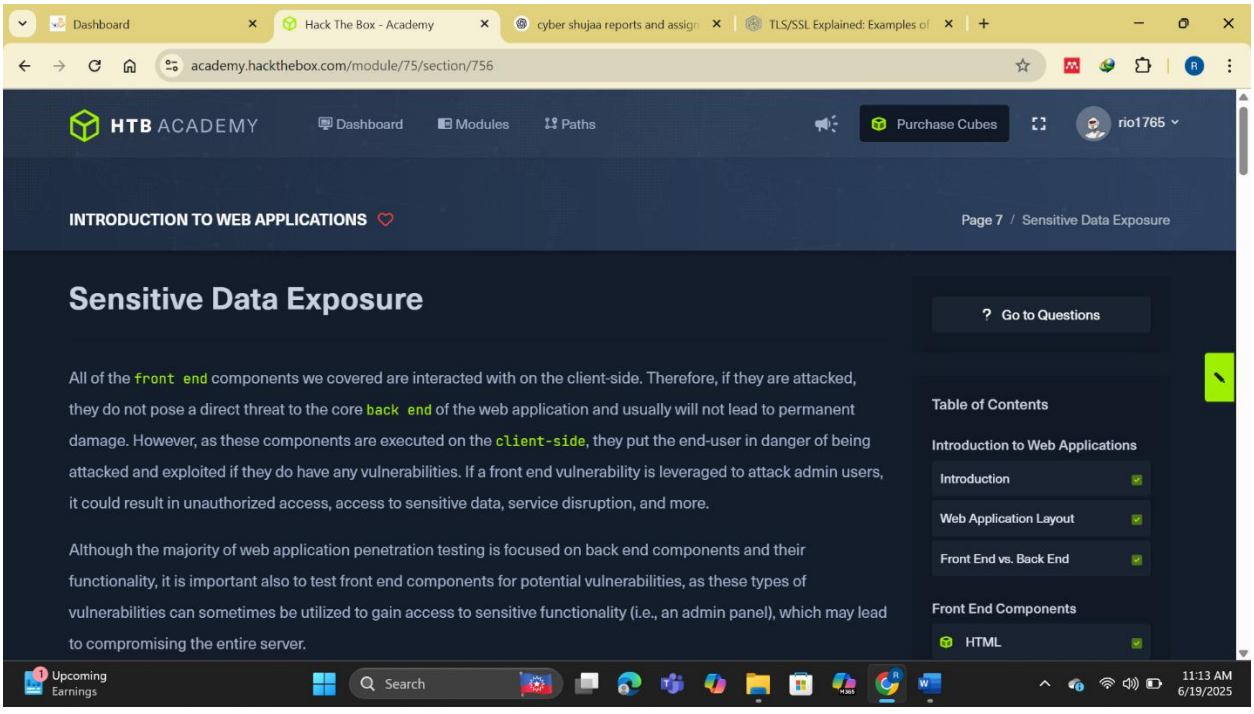

Fig 1.8 sensitive data exposure

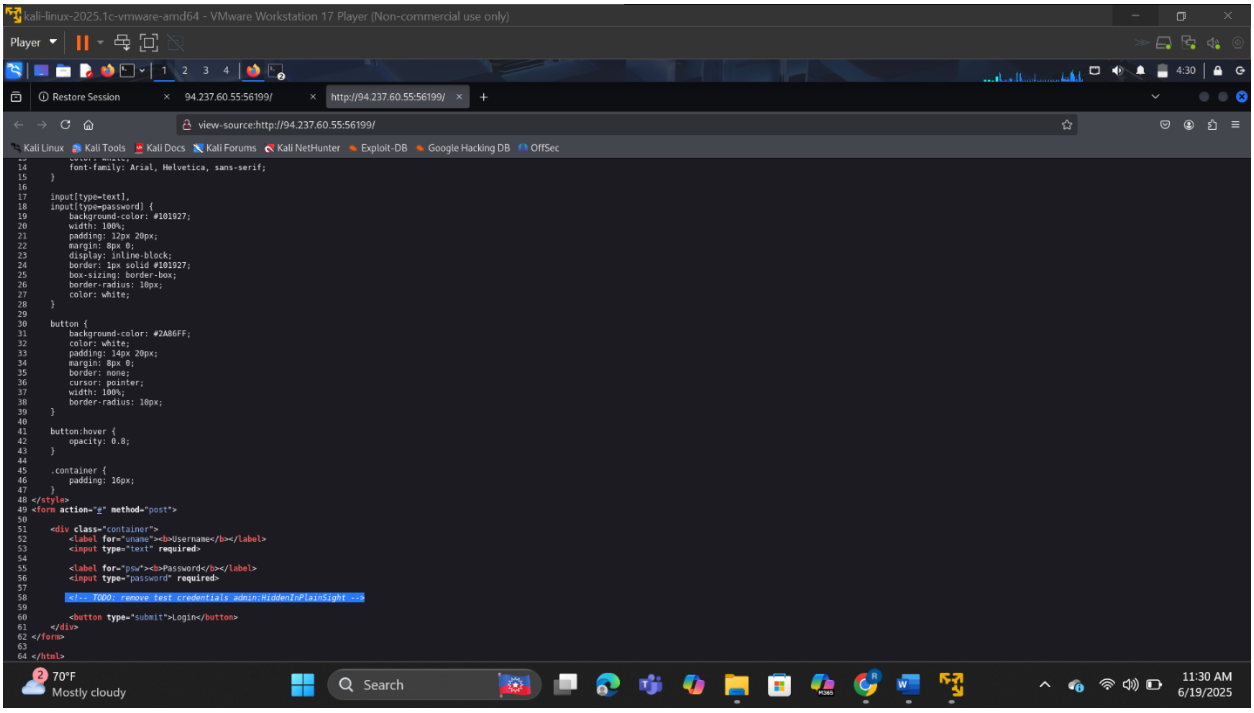

Fig 1.9 Finding exposed credentials in the login page

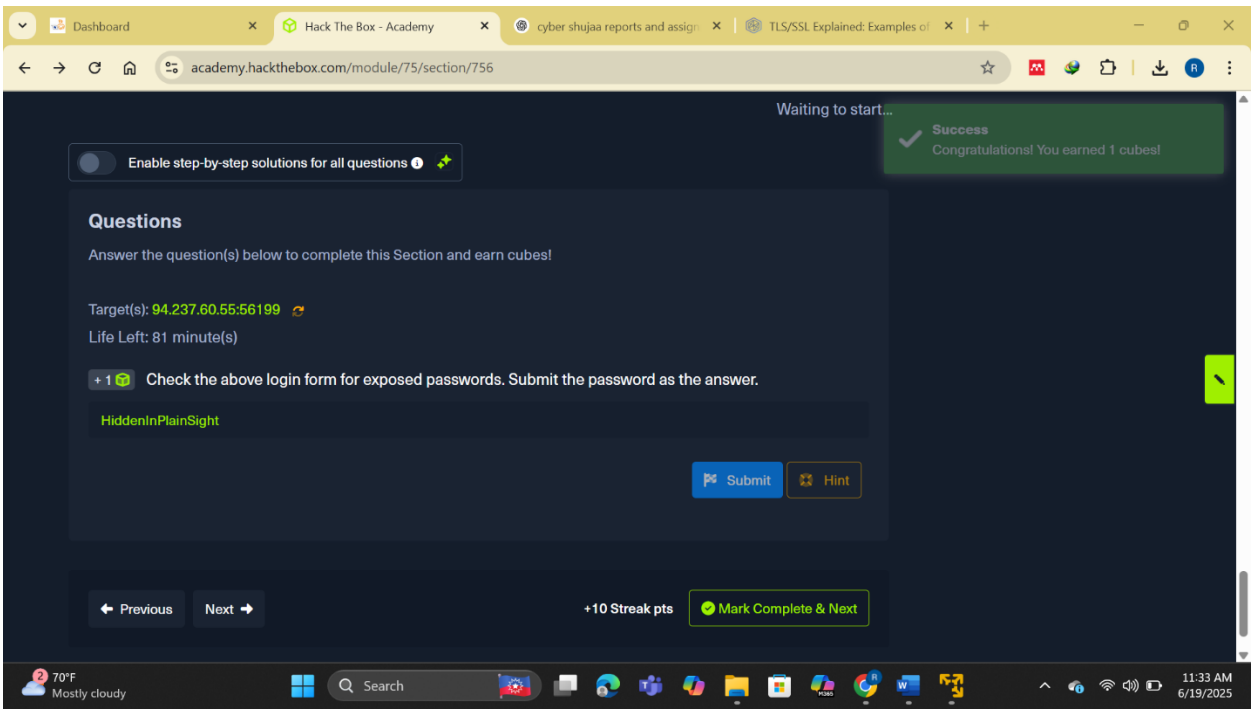

Fig 2.0 the exposed credential

HTML Injection

While learning about HTML Injection, I realized how critical it is to validate and sanitize user input not just on the back end, but also directly on the front end of a web application. Often, developers assume that all input is handled server-side and neglect the fact that many inputs can be processed, rendered, and even displayed entirely by the browser before the back end is involved. This opens the door to HTML Injection attacks—where unfiltered input is displayed as actual HTML on the page—making the user interface vulnerable to manipulation and users themselves open to phishing, data theft, or defacement.

To understand this better, I worked with a simple example where a web page displayed a button that, when clicked, asked me to enter my name and then displayed that name on the screen. The source code for the page looked like this:

```
<html>
<body>
<button onclick="inputFunction()">Click to enter your name</button>
<p id="output"></p>
<script>
function inputFunction() {
var input = prompt("Please enter your name", "");
if (input != null) { document.getElementById("output").innerHTML = "Your name is " + input; }
}
</script>
</body>
</html>.
```

Here, I noticed that the page used innerHTML to insert the user's input directly into the DOM without any form of sanitization. This meant that any HTML tags I included in my input would be treated as actual HTML, not just plain text.

To test this, I entered the payload <style> body { background-image: url('https://academy.hackthebox.com/images/logo.svg'); } </style>. As soon as I submitted it, the entire background of the web page changed to the Hack The Box Academy logo. This simple example demonstrated the concept of HTML Injection in action—my input was not treated as raw text but was executed as part of the page's layout.

This type of vulnerability, although appearing cosmetic in this case, can have serious consequences if exploited maliciously. An attacker could inject fake login forms, misleading ads, or links to phishing sites—especially if the application is accessed by an admin or a user with elevated privileges. This could lead to stolen credentials or loss of control over parts of the system.

What I learned is that developers must never assume the browser will "handle things safely" on its own. Instead, all user input must be validated, sanitized, or escaped properly, especially when rendered into the DOM. Even something as simple as displaying a name on the screen can become an attack vector if done carelessly. For penetration testers like me, testing for HTML injection becomes an essential part of analyzing front-end security, because it offers a straightforward yet powerful way to evaluate how securely an application handles client-side rendering of user input.

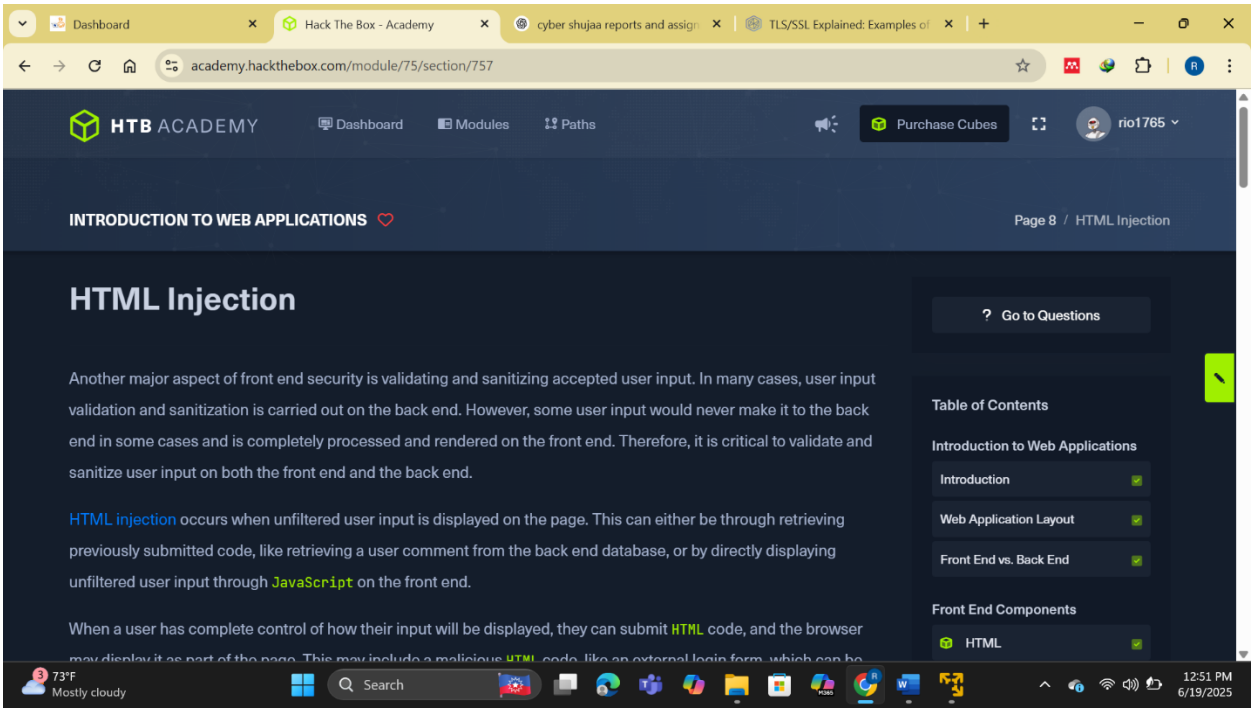

Fig 2.1 html injection

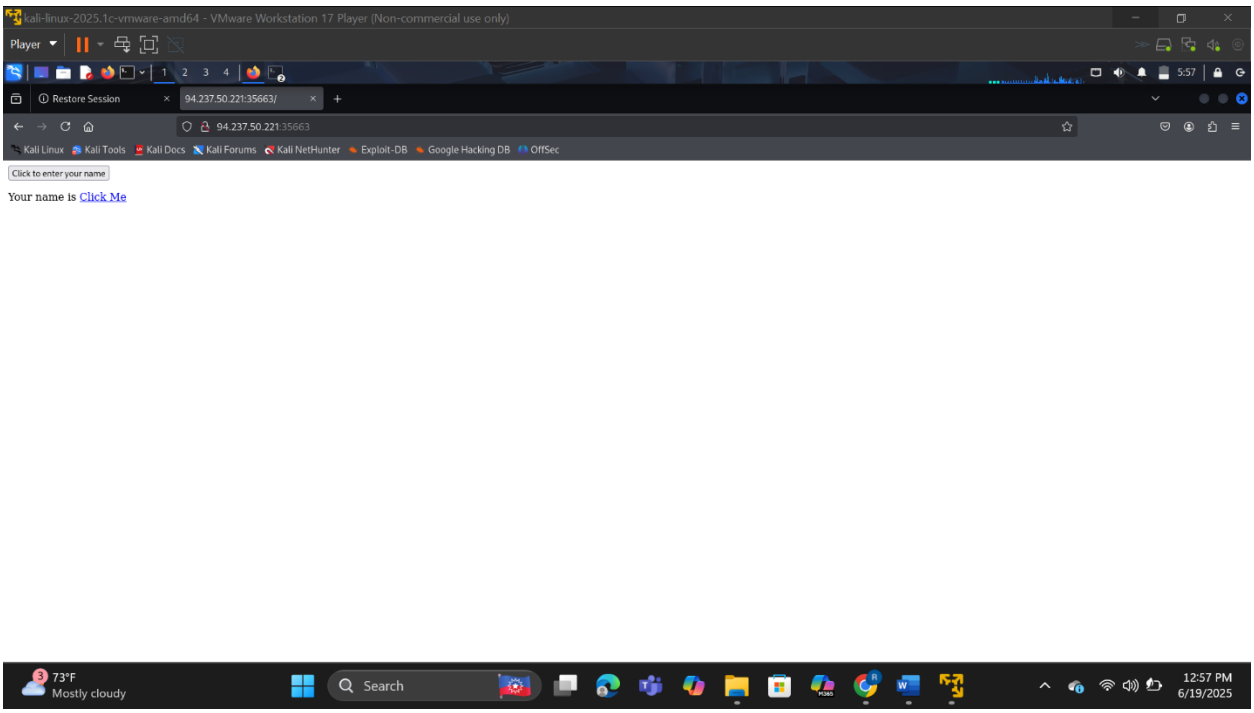

Fig 2.2 finding the name when we inject the payload

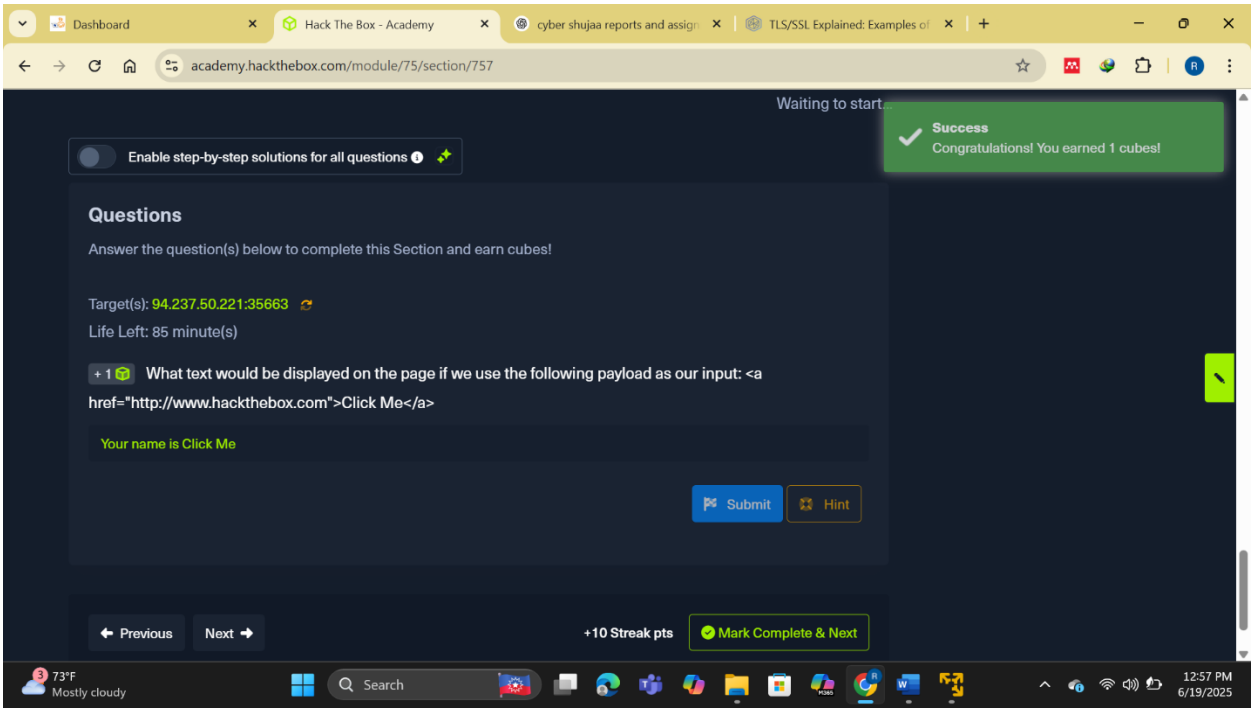

Fig 2.3 answer to the name found after injecting the payload

Cross-Site Scripting (XSS)

While exploring the topic of Cross-Site Scripting (XSS), I deepened my understanding of how HTML Injection vulnerabilities can escalate into more serious security threats. I learned that XSS attacks are similar in structure to HTML Injection but far more dangerous because they involve the injection of JavaScript code, which can be executed directly in the victim's browser. This opens up a wide range of possibilities for attackers—from stealing session cookies to hijacking accounts or even launching full client-side attacks.

There are three main types of XSS I familiarized myself with: Reflected XSS, which occurs when input is immediately reflected back to the user (like in a search bar); Stored XSS, where malicious input is stored in the backend (such as in a comment section); and DOM XSS, where JavaScript dynamically updates the DOM based on user input. In this exercise, I focused specifically on DOM XSS.

To test this on the vulnerable web page provided, I crafted a simple but effective payload: #"><img src=/ onerror=alert(document.cookie)>. This payload leverages the img tag's onerror event handler to execute JavaScript code when the browser fails to load the image source. As expected, after entering this payload and confirming the prompt, a JavaScript alert box appeared on the screen displaying my browser's cookie value. This confirmed that the application was vulnerable to DOM-based XSS.

The payload worked because the input was rendered directly into the HTML DOM without being sanitized or escaped, allowing my JavaScript code to run as part of the document. The function alert(document.cookie) accessed the browser's document.cookie object and displayed its contents, which often include session identifiers. This information can be stolen and sent to an attacker-controlled server to hijack user sessions.

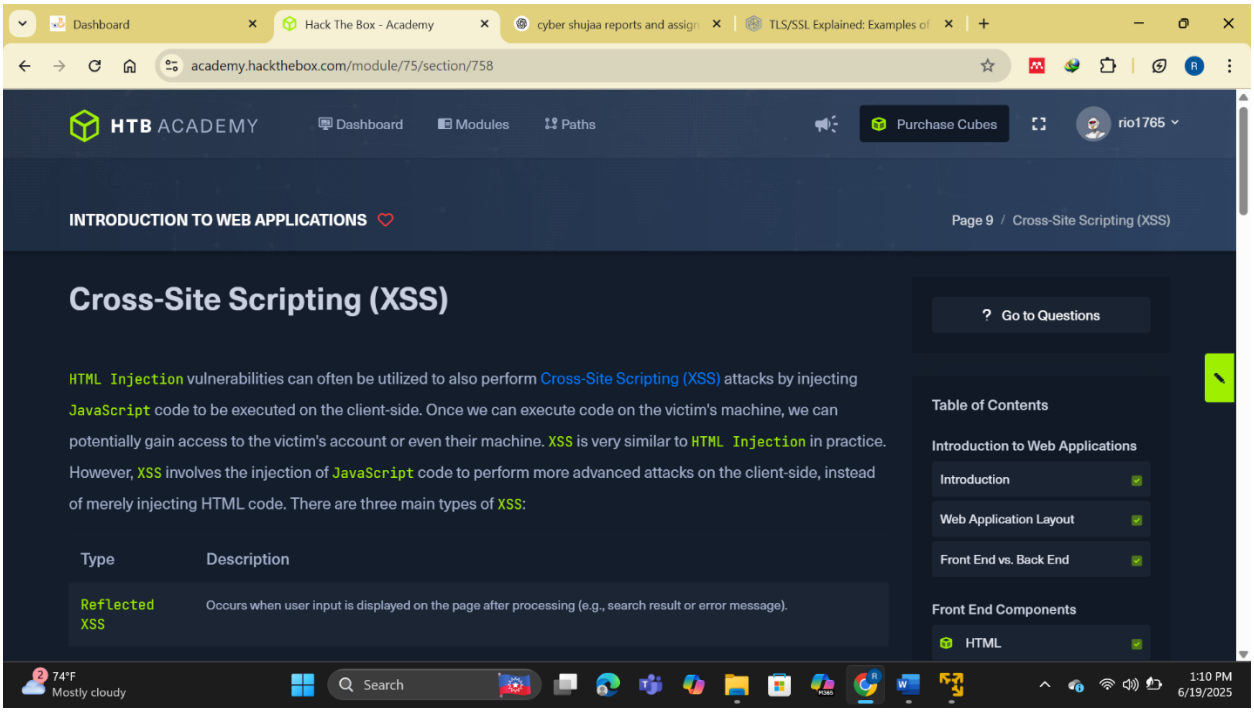

Fig 2.4 cross-site scripting

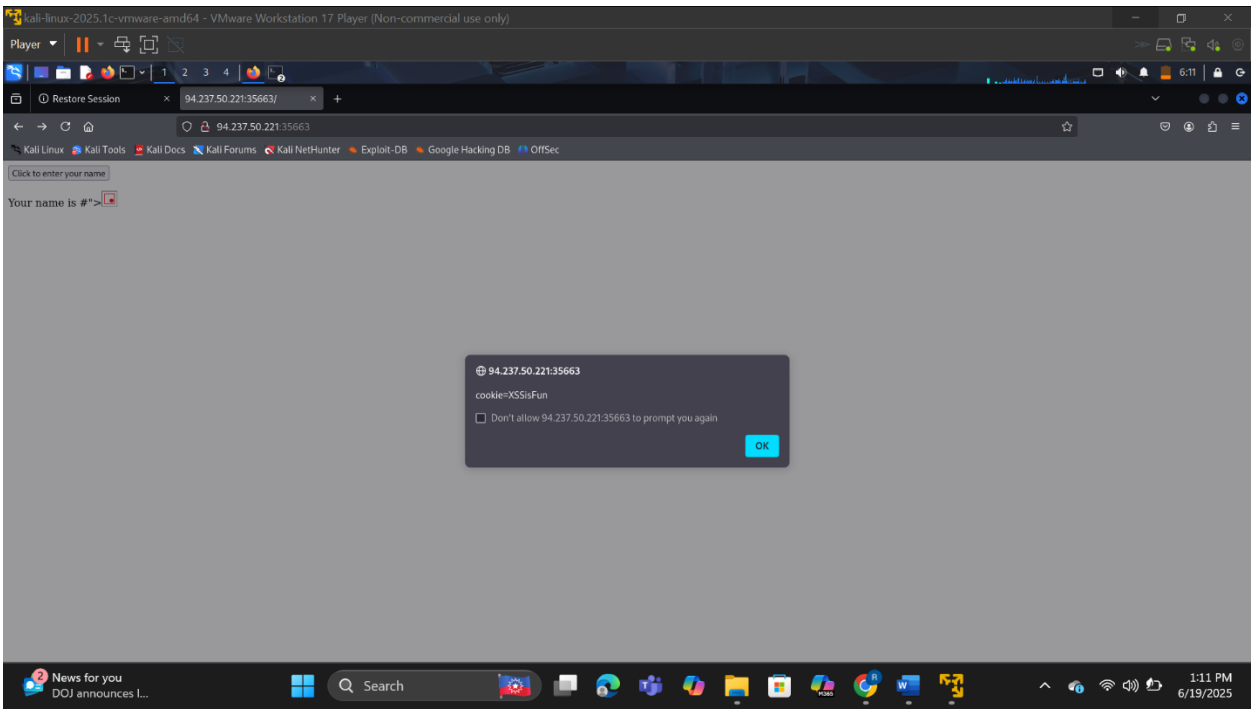

Fig 2.5 using java script to obtain cookie(through xss)

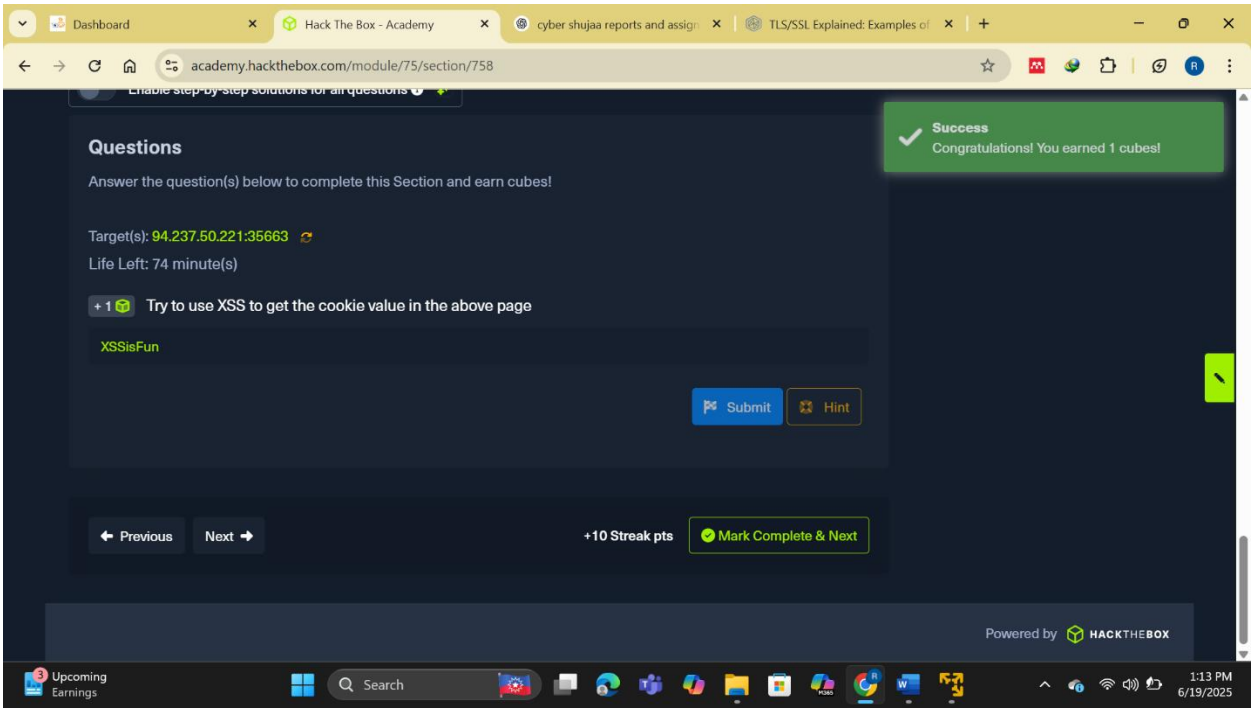

Fig 2.6 cookie value of the page

Cross-Site Request Forgery (CSRF)

From this section, I deepened my understanding of Cross-Site Request Forgery (CSRF), a serious front-end vulnerability caused by unfiltered user input. I learned that CSRF attacks often exploit XSS vulnerabilities to send forged requests to a web application that a victim is already authenticated to, making the application believe that the actions are performed by the legitimate user. This allows attackers to perform privileged operations like changing passwords, updating account details, or even executing administrative tasks, all without the victim's consent or awareness.

I discovered that a typical CSRF scenario involves an attacker planting a malicious JavaScript payload within a web page, which executes automatically once the user (already logged into the target application) views the page. For example, an attacker could place the payload "><script src=//www.example.com/exploit.js></script> into a vulnerable comment section. When the victim loads the page, this script is fetched from the attacker's server and executed in the context of the victim's session. If exploit.js is crafted to mimic a legitimate password change request, the script could silently change the victim's password to one known to the attacker, granting them access to the victim's account.

Understanding this, I realized the importance of how attackers can craft such an exploit.js file. It requires a deep knowledge of the web application's internal password reset mechanisms and APIs. Once the password is changed silently via the victim's session, the attacker simply logs in using the new credentials.

The section also emphasized the need for robust security practices to prevent such attacks. I learned that proper user input handling must include both sanitization—removing or escaping special and non-standard characters—and validation—ensuring that inputs match expected formats like valid emails or

strings. However, input filtering alone is not enough. Defense-in-depth is key: implementing anti-CSRF tokens (unique for each session or request), using HTTP cookie flags like SameSite=Strict, and requiring sensitive actions to be confirmed by entering a password are all crucial defensive layers.

Moreover, modern browsers offer built-in protections against certain types of XSS, and many web applications now rely on frameworks and security headers that can reduce CSRF exposure. Still, I understood that even with those in place, determined attackers may find creative ways to bypass them. Therefore, CSRF and XSS remain highly relevant threats and must be considered from both the development and security auditing perspectives.

This section reinforced the idea that application security should be built into the design from the beginning—not treated as an afterthought. I've come to appreciate the role of secure coding, layered defenses, and the critical thinking required to spot and mitigate subtle but devastating vulnerabilities like CSRF.

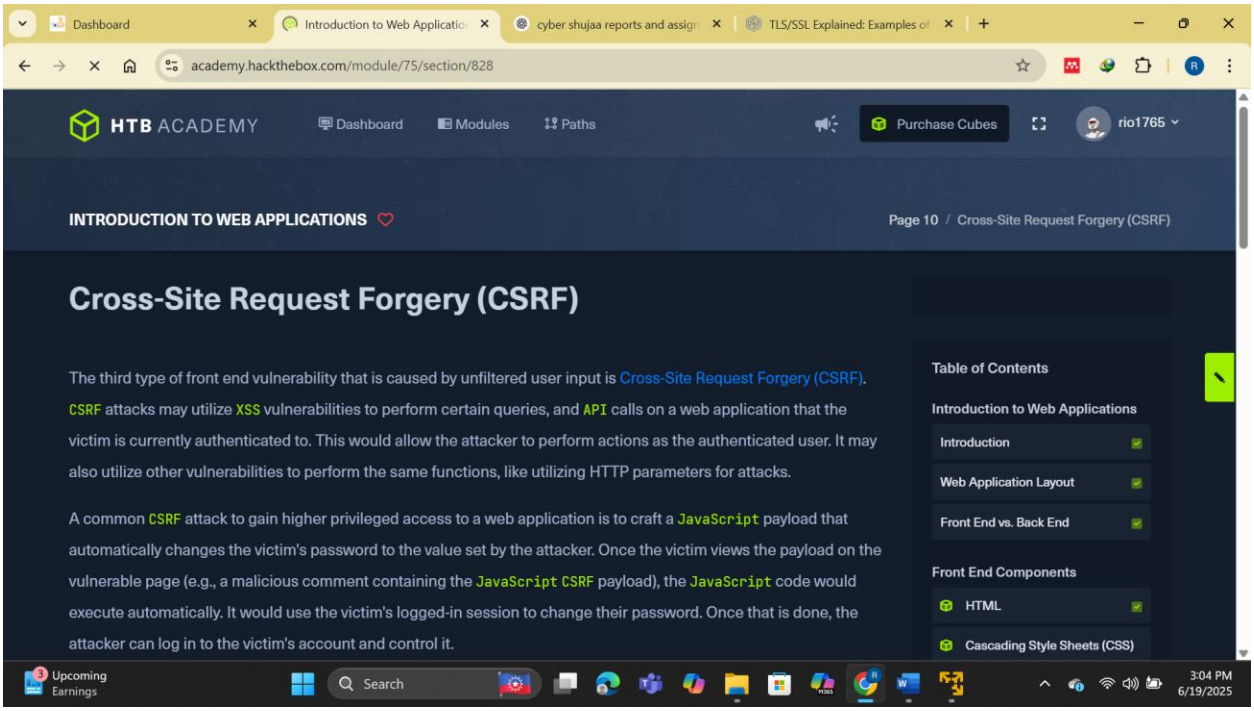

Fig 2.7 cross site request forgery

Back End Servers

In this section, I learned that the back-end server is the powerhouse of any web application, responsible for running all the critical processes behind the scenes. It fits into the data access layer and is where the logic, computation, and data processing happen. The back-end server hosts essential components like the **web server**, **database**, and **development framework**, all working together to support the functionality of the web application accessed by users through their browsers.

I explored how these servers can be running various operating systems and server software, such as **Apache**, **NGINX**, or **IIS** for web serving, paired with programming languages like **PHP**, **C#**, or **Java**,

and connected to databases such as **MySQL**, **MS SQL**, or **Oracle**. What really stood out to me were the different *web solution stacks* used to combine these elements in specific configurations. For example, the **LAMP** stack includes Linux, Apache, MySQL, and PHP, which is one of the most widely used setups for Linux-based servers. On the other hand, **WAMP** uses Windows instead of Linux and is a preferred choice for Windows environments. Other variations include **WINS** (Windows, IIS, .NET, SQL Server), **MAMP** (macOS, Apache, MySQL, PHP), and **XAMPP** (a cross-platform version using Apache, MySQL, and PHP/Perl).

Besides software, I realized how important the **hardware** of the back-end server is. The responsiveness and scalability of a web application depend greatly on how robust and powerful the hardware is. While some web apps may be hosted on a single physical server, most large-scale applications are distributed across multiple servers or run in virtualized environments like **cloud infrastructure** or **containers**, often managed through data centers. This allows them to scale efficiently and stay highly available even unde

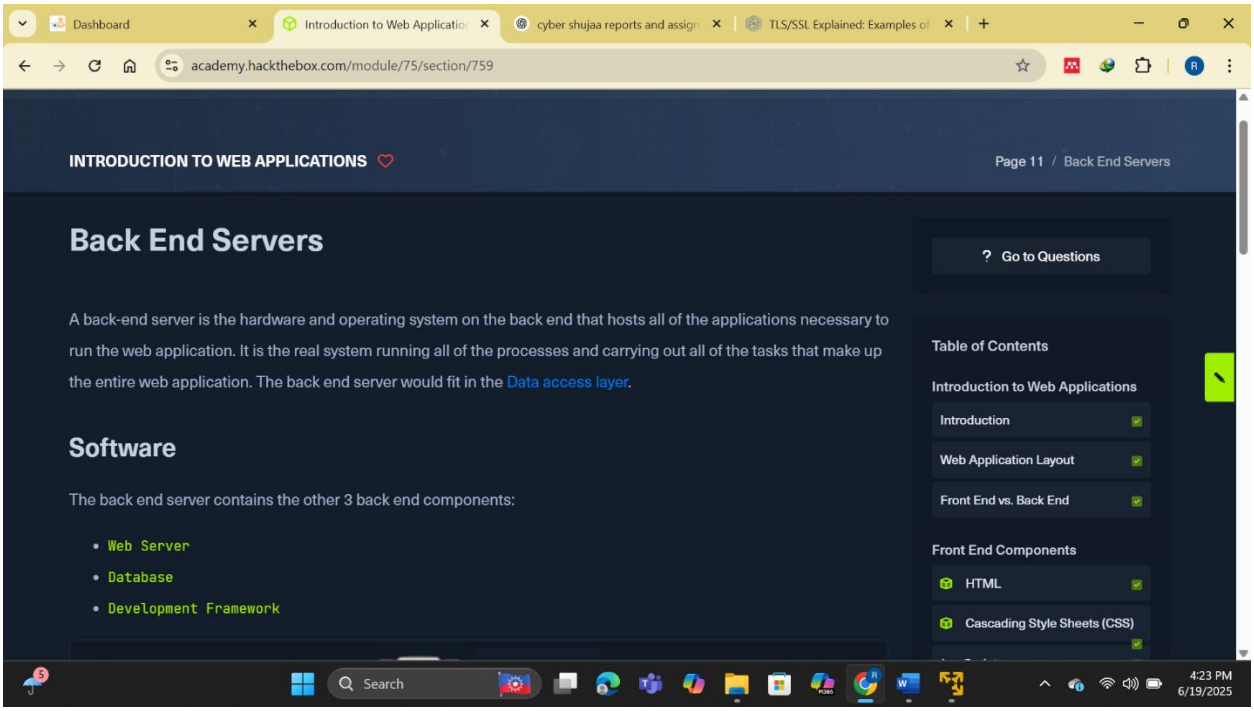

Fig 2.8 backend servers

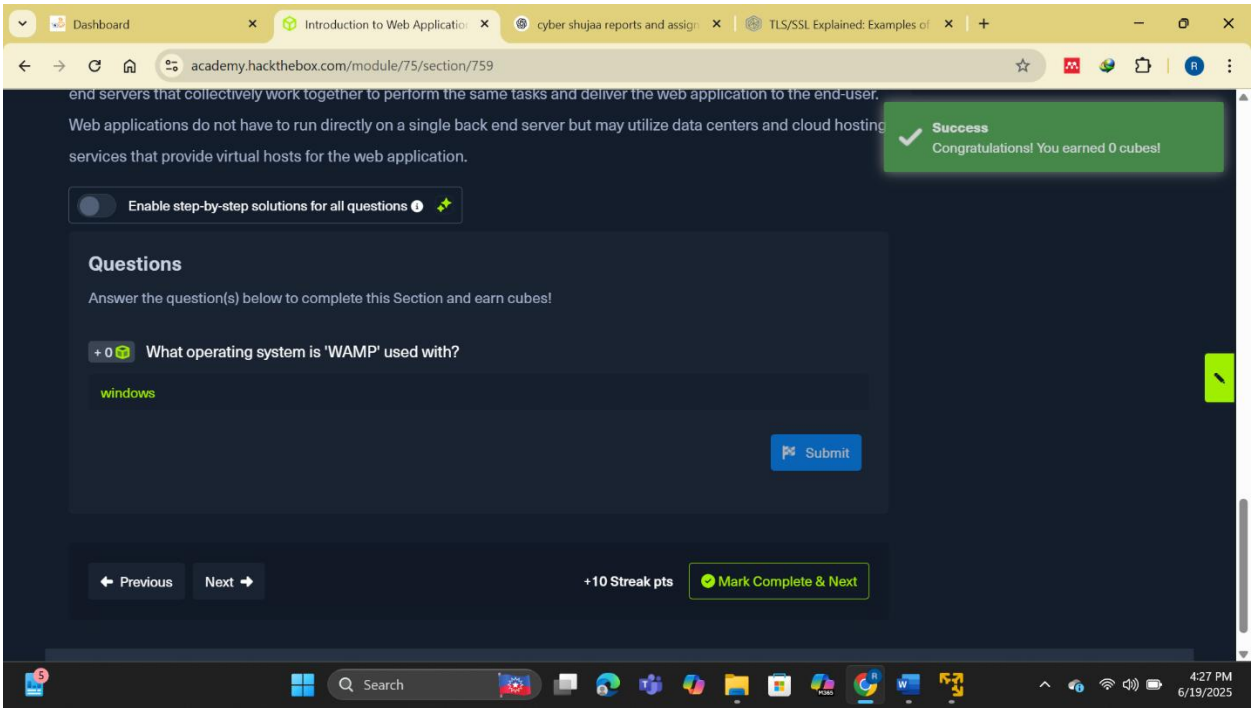

Fig 2.9 answers to question on backend

Web Servers

I learned about the role and functionality of **web servers** within the backend of a web application. A web server is an application running on the backend server, responsible for handling **HTTP requests** from client-side browsers and routing those requests to the correct backend resources, before sending the appropriate **HTTP responses** back to the users. These servers typically operate over **TCP ports 80 and 443**, handling both unencrypted (HTTP) and encrypted (HTTPS) traffic respectively.

I understood the importance of the **HTTP response codes** that web servers return to indicate the status of a client's request. For instance, I encountered common status codes such as **200 OK** for successful requests, **404 Not Found** for pages that don't exist, **403 Forbidden** when access is restricted, and **500 Internal Server Error** for server-side issues. These codes provide essential feedback for both users and developers to understand how the server is interpreting the request. I practiced using the **curl** utility to request page headers with curl -I https://academy.hackthebox.com, which returned metadata like HTTP/2 200 and content-type: text/html; charset=UTF-8, and I also retrieved the raw HTML content of the page using a simple curl https://academy.hackthebox.com.

The section also introduced me to three of the most widely used web servers:

1. **Apache** – the most prevalent web server, powering over 40% of websites globally. It's open-source, cross-platform, and supports multiple languages like PHP, Python, Perl, and more. I learned that Apache uses modules (like mod_php) to support scripting languages and is praised for its flexibility, documentation, and strong community support.

2. **NGINX** – the second most popular server, known for its high performance and efficient handling of concurrent connections due to its **asynchronous architecture**. NGINX is preferred for high-traffic sites, with companies like Google, Facebook, and Netflix relying on it. Its resource efficiency makes it a strong choice for enterprise applications.

3. **IIS (Internet Information Services)** – developed by Microsoft and primarily used in **Windows Server** environments. It is closely integrated with **Active Directory** and supports web applications built on the **.NET framework**, though it can also host PHP and FTP services. I discovered that sites like Microsoft.com, Skype, and Stack Overflow use IIS due to its compatibility with Microsoft's infrastructure.

Beyond these three, I also learned about other specialized web servers like **Apache Tomcat** for Java applications and **Node.js** for JavaScript-based backend services. These tools highlight the diverse technology stack options available depending on a project's specific requirements.

This exploration deepened my understanding of how a web server acts as the critical communication point between client and server, not just responding with data, but also enforcing access control, handling load, and enabling extensibility through modules or scripts. This foundational knowledge is essential for both secure development and penetration testing of modern web applications.

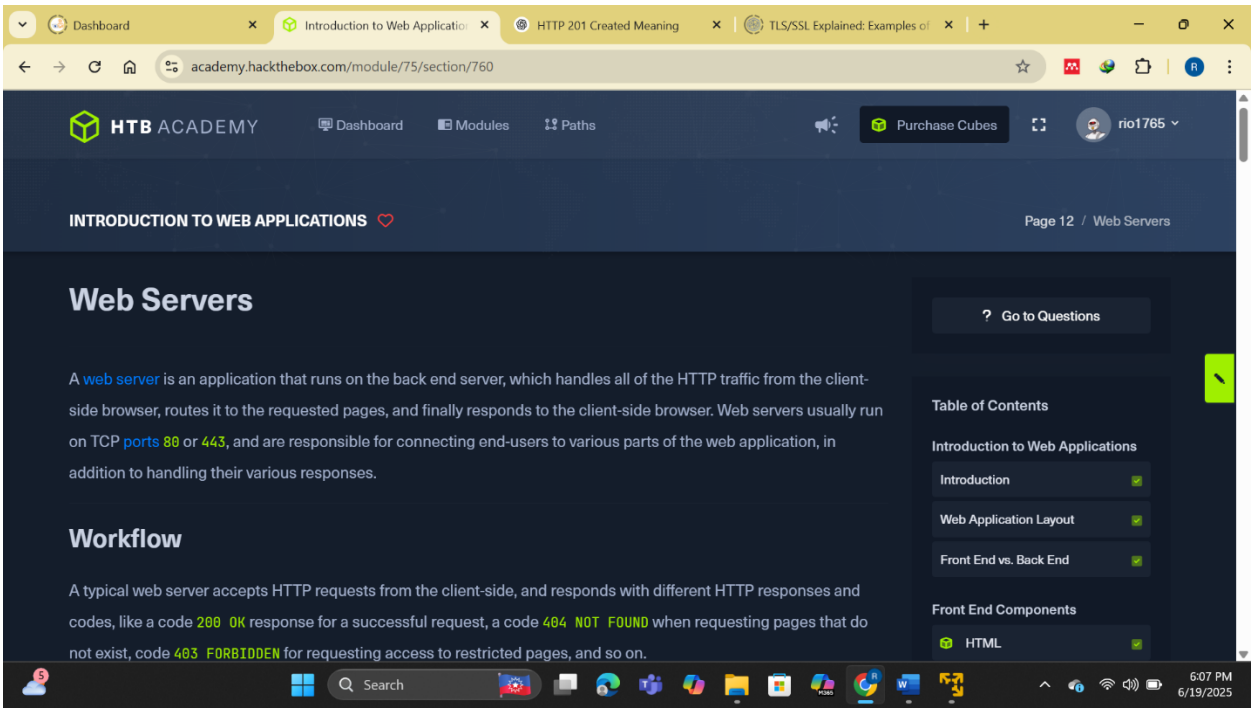

Fig 3.0 webserver

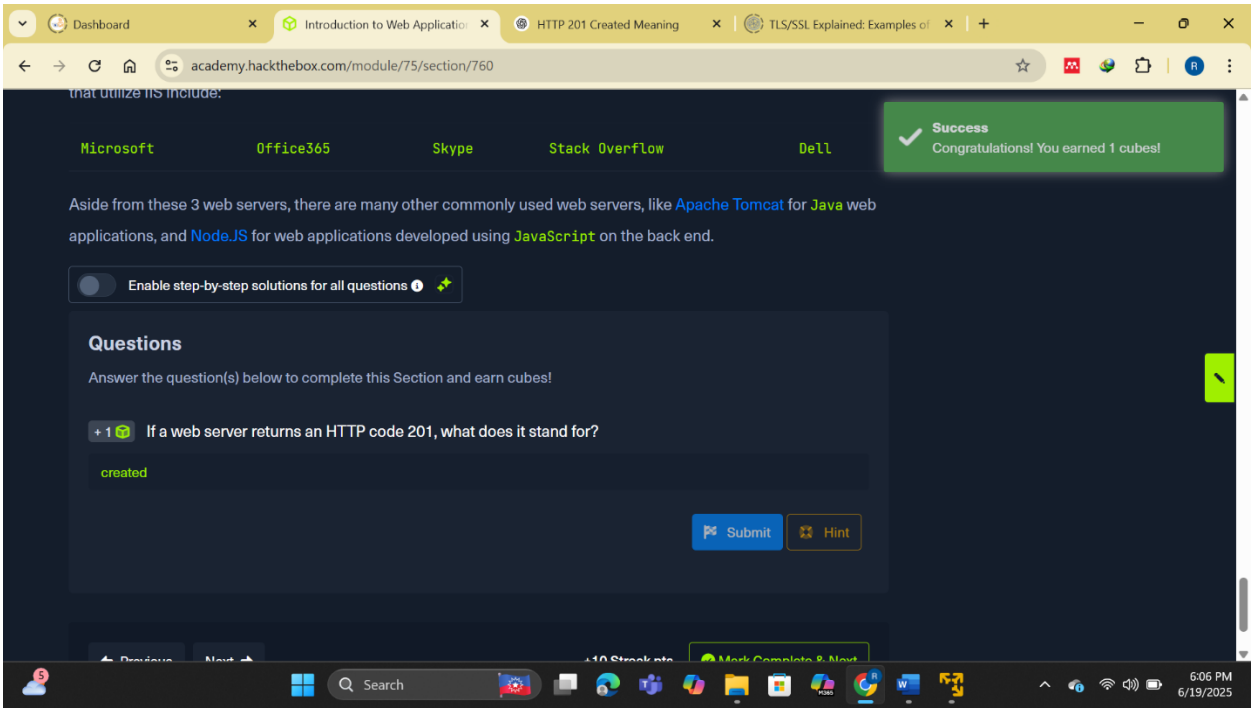

Fig 3.1 answer to question on we server

Databases

In this section, I learned about the essential role of **databases** in the functioning of web applications. Databases are used on the **backend** to store everything from images and files to user accounts, passwords, posts, and application settings. They enable the dynamic and personalized content that defines modern web applications, allowing information to be easily stored and retrieved.

I explored the two primary types of databases used in web development: **Relational (SQL)** and **Non-relational (NoSQL)**. Relational databases store data in **structured tables** composed of rows and columns. These tables can be linked together through **keys**, forming what's called a **schema**. For instance, a users table with fields like id, username, first_name, and last_name can be linked to a posts table via user_id to associate posts with their authors. This makes retrieving data highly efficient. Some of the common relational databases I reviewed include **MySQL**, **MSSQL**, **Oracle**, and **PostgreSQL**.

In contrast, **NoSQL databases** do not rely on fixed schemas or structured tables. Instead, they use flexible storage models such as **Key-Value**, **Document-Based**, **Wide-Column**, and **Graph** models. This makes them highly scalable and ideal for storing unstructured or semi-structured data.

Each post has a unique key and associated JSON object. NoSQL options like **MongoDB**, **ElasticSearch**, and **Apache Cassandra** provide powerful alternatives when traditional relational databases aren't a good fit, especially in high-volume, less-structured environments. I also learned how databases are integrated into web applications.

I also understood that poor implementation of such logic could introduce serious vulnerabilities, like **SQL Injection**. Therefore, security must always be part of database integration.

Ultimately, this section helped me appreciate the importance of selecting the right database model for a given application and implementing secure, scalable solutions for data management. Whether using relational structures for structured data or NoSQL systems for more flexible use cases, understanding how databases interact with backend and frontend components is key to building robust, secure web applications.

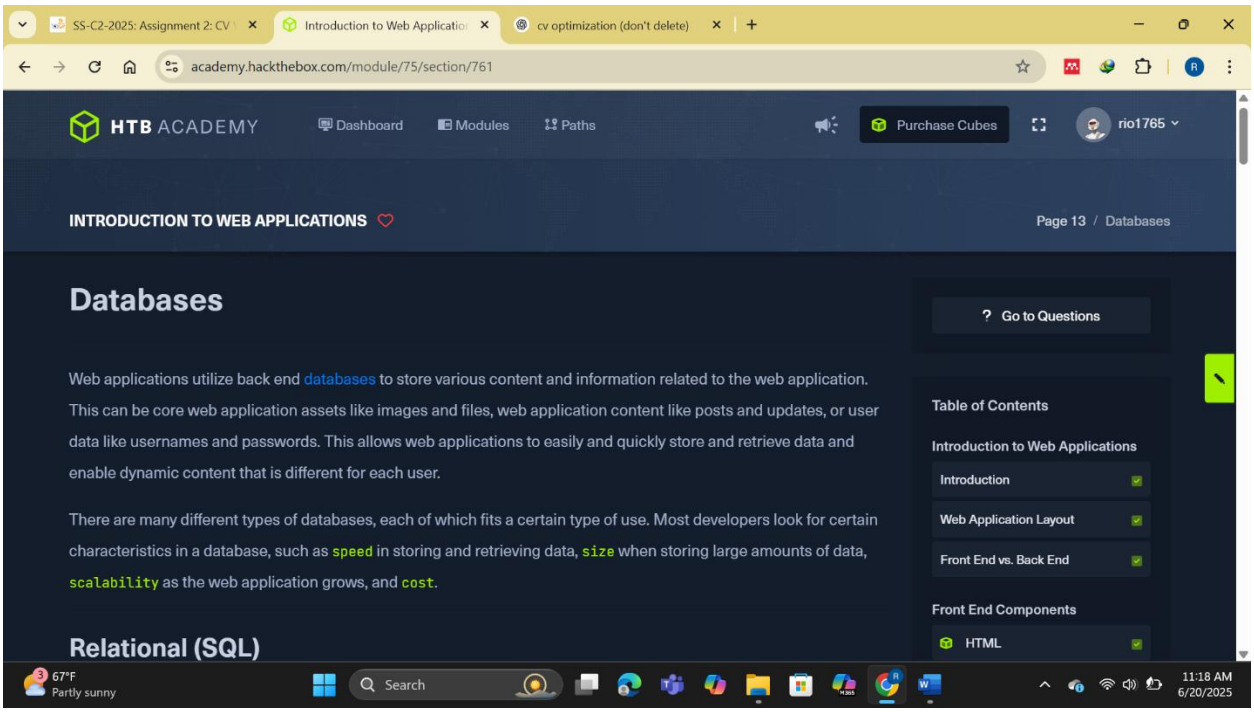

Fig 3.2 databases

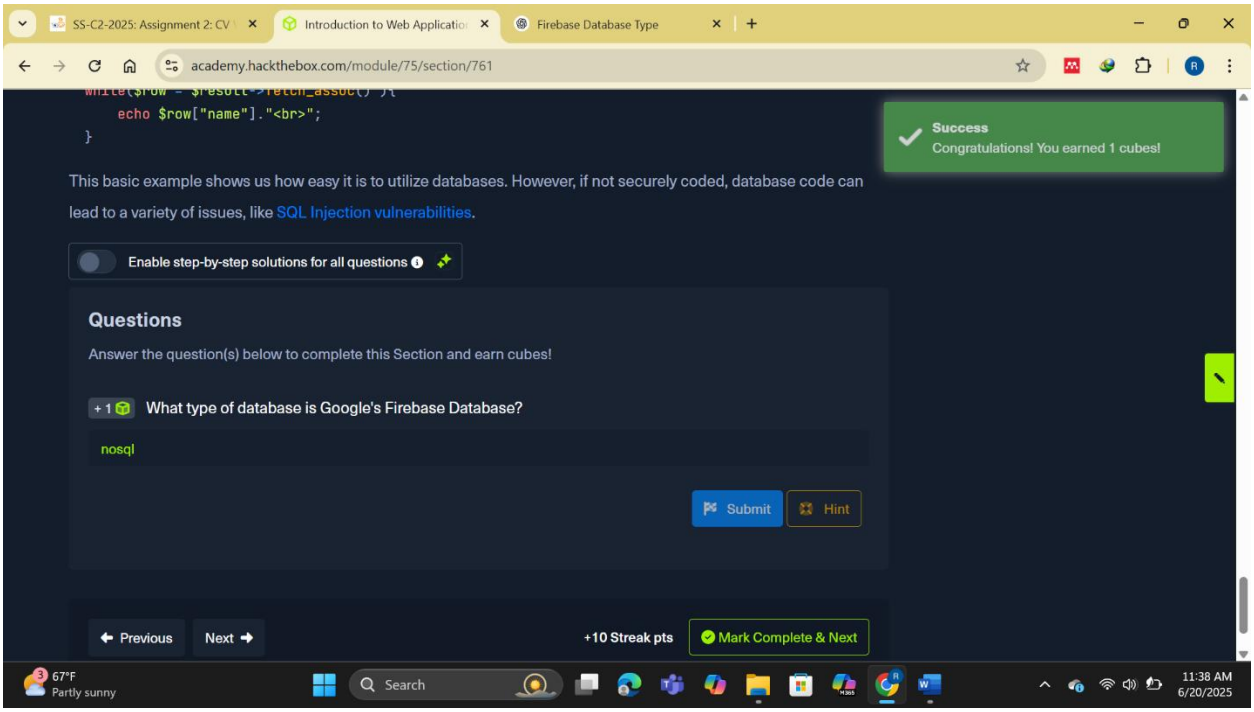

Fig 3.3 answer to question on databases

Development Frameworks & APIs

In this section, I learned how **development frameworks** and **APIs** significantly streamline the creation and functionality of web applications. Rather than building everything from scratch, developers now leverage powerful web development frameworks that offer ready-made solutions for common features like user registration, authentication, routing, and database interaction.

Some of the most widely used development frameworks I explored include **Laravel** for PHP, which is lightweight and favored by startups; **Express** for Node.js, used by major companies like Uber and PayPal; **Django** for Python, adopted by tech giants like Google and Instagram; and **Rails** for Ruby, used by companies like GitHub and Airbnb. These frameworks abstract much of the complexity involved in backend development, allowing faster and more secure application development.

I also delved into **APIs** (Application Programming Interfaces), which serve as a bridge between front-end and back-end components of a web application. APIs allow the frontend to make requests to the backend to perform certain actions—like fetching user data or posting a message—and then display the response to the user. These interactions are often made using **HTTP request parameters**, including **GET** and **POST** methods.

SOAP is useful when stateful operations are needed, such as updating an application's session state or sending complex data. However, its verbosity makes it less efficient for simpler tasks.

REST (Representational State Transfer), on the other hand, is lightweight and widely adopted in modern web applications. It operates over standard HTTP methods and usually returns **JSON** responses, making it easier to parse and integrate.

RESTful APIs typically use HTTP verbs to define actions:

- GET to retrieve data,
- POST to create data,
- PUT to update or replace data,
- DELETE to remove data.

Understanding how APIs handle data, and how development frameworks integrate these interactions seamlessly, is essential for building and attacking modern web applications. This knowledge is especially crucial in penetration testing, where inspecting API endpoints and HTTP request behavior can reveal hidden vulnerabilities or misconfigurations.

Overall, this section gave me a practical grasp of how APIs power modern web functionality and how frameworks help developers build scalable, secure, and maintainable applications efficiently.

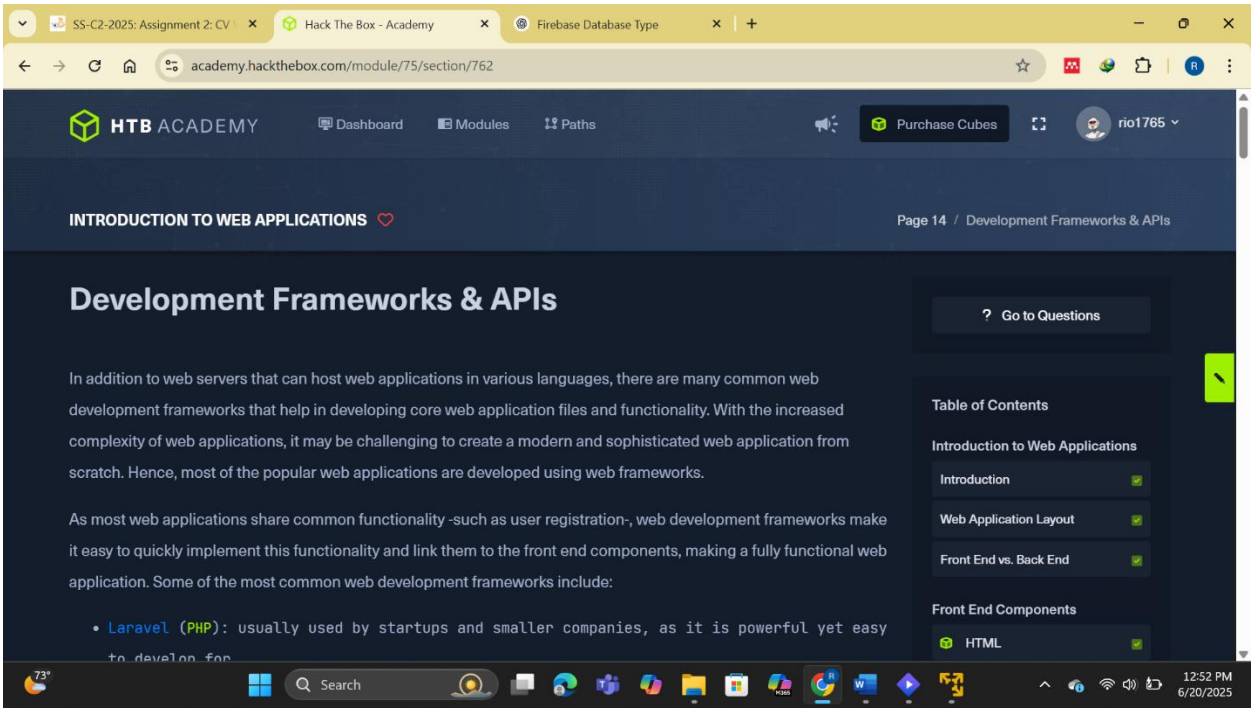

Fig 3.4 development frameworks and api

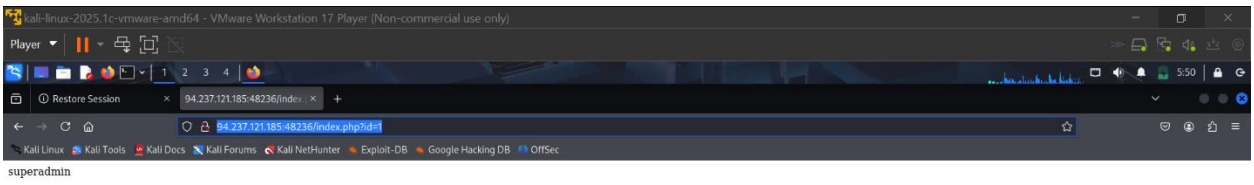

Fig 3.5 using get request to find the name

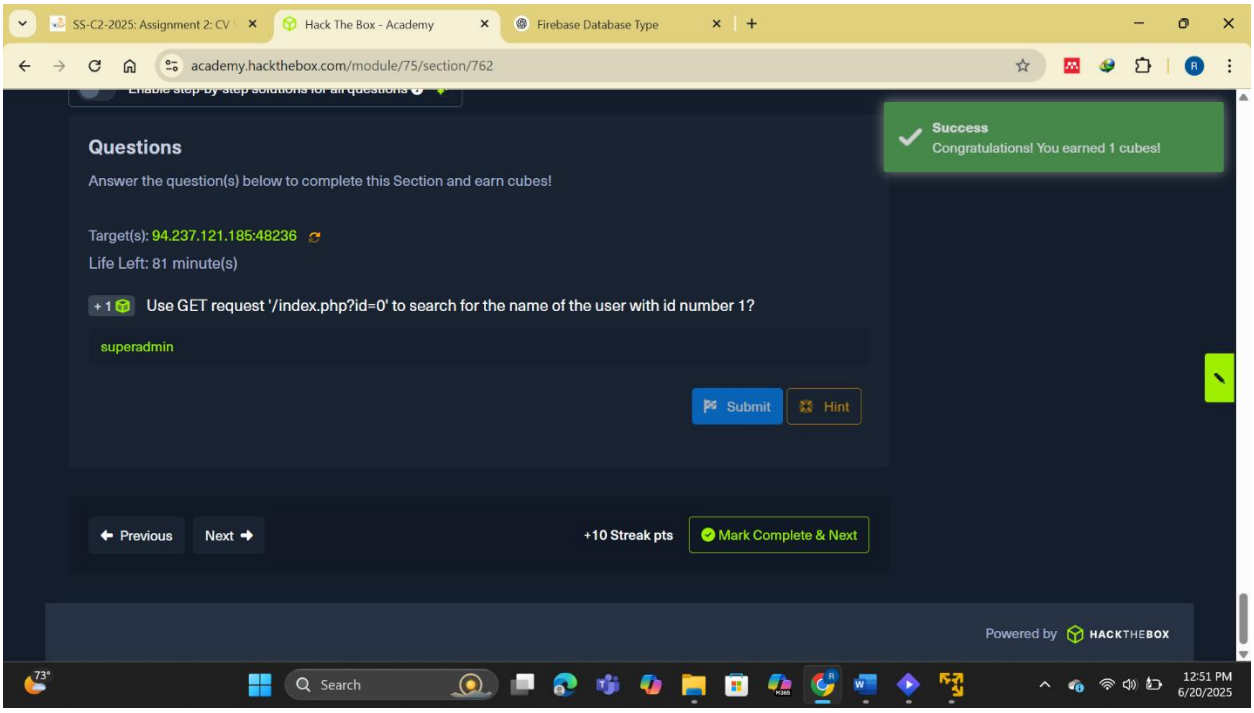

Fig 3.6 answer to question on development frameworks and api

Common Web Vulnerabilities

In this section, I learned about **some of the most common vulnerabilities in web applications**, many of which are part of the **OWASP Top 10**. These vulnerabilities often occur not due to flaws in the core design of the application itself, but from **insecure coding practices, poor user input handling, or configuration mistakes** made by developers.

One of the first vulnerabilities I studied was **Broken Authentication and Broken Access Control**. These occur when users can either bypass login mechanisms altogether or access privileged areas without authorization. For example, I learned that in the **College Management System 1.2**, authentication can be bypassed by injecting ' or 0=0 # into the email input field and submitting any password. This form of SQL logic injection tricks the system into authenticating a user without credentials.

Next, I explored **Malicious File Upload** vulnerabilities, where a poorly protected file upload feature allows an attacker to upload harmful scripts. If a developer doesn't check file types properly or is tricked by **double extensions** like shell.php.jpg, the attacker might successfully upload a **malicious PHP script**. This can give them a backdoor into the web server. A real-world example of this is the **WordPress Plugin Responsive Thumbnail Slider 1.0**, which can be exploited using Metasploit.

Another dangerous vulnerability is **Command Injection**, where user input is included in system-level commands. If not properly sanitized, attackers can append their own commands. For example, in the **Plainview Activity Monitor plugin**, inputting an IP value like 127.0.0.1 | whoami could cause the server to run both commands, effectively giving the attacker remote shell access.

I also revisited **SQL Injection (SQLi)**, a classic and still very prevalent vulnerability. Insecure SQL queries like:

php

CopyEdit

$query = "select * from users where name like '%$searchInput%'";

can be manipulated if $searchInput includes SQL code. This could be used to **bypass login**, **dump sensitive data**, or even **gain access to the underlying server**. I saw that **College Management System 1.2** also suffers from such a vulnerability, and attackers can exploit it using simple SQL tricks that always return true, effectively bypassing authentication.

Finally, I was reminded that vulnerabilities like these don't just apply to unknown applications. Even large, publicly available applications can be misconfigured or poorly maintained, making them exploitable. It's crucial for me, as a budding penetration tester, to deeply understand each of these issues. They are recurring themes in real-world assessments, and having a strong foundation in them will set me apart in any InfoSec role.

I also researched that **CVE-2014-6271**—commonly known as **Shellshock**—belongs to the category of **Command Injection vulnerabilities**, as it allows arbitrary commands to be injected into and executed by Bash. This vulnerability is a perfect example of why it's critical to validate and sanitize all user inputs, especially in scripts or system-level processes.

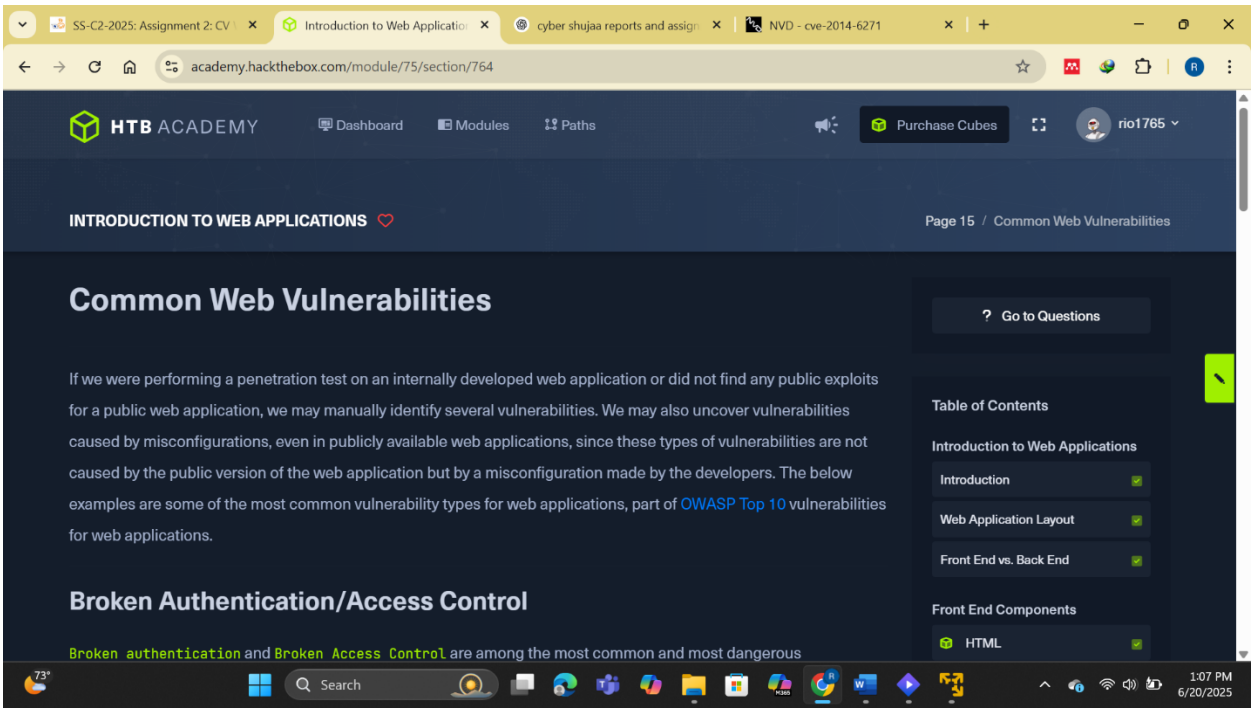

Fig 3.7 common vulnerabilities

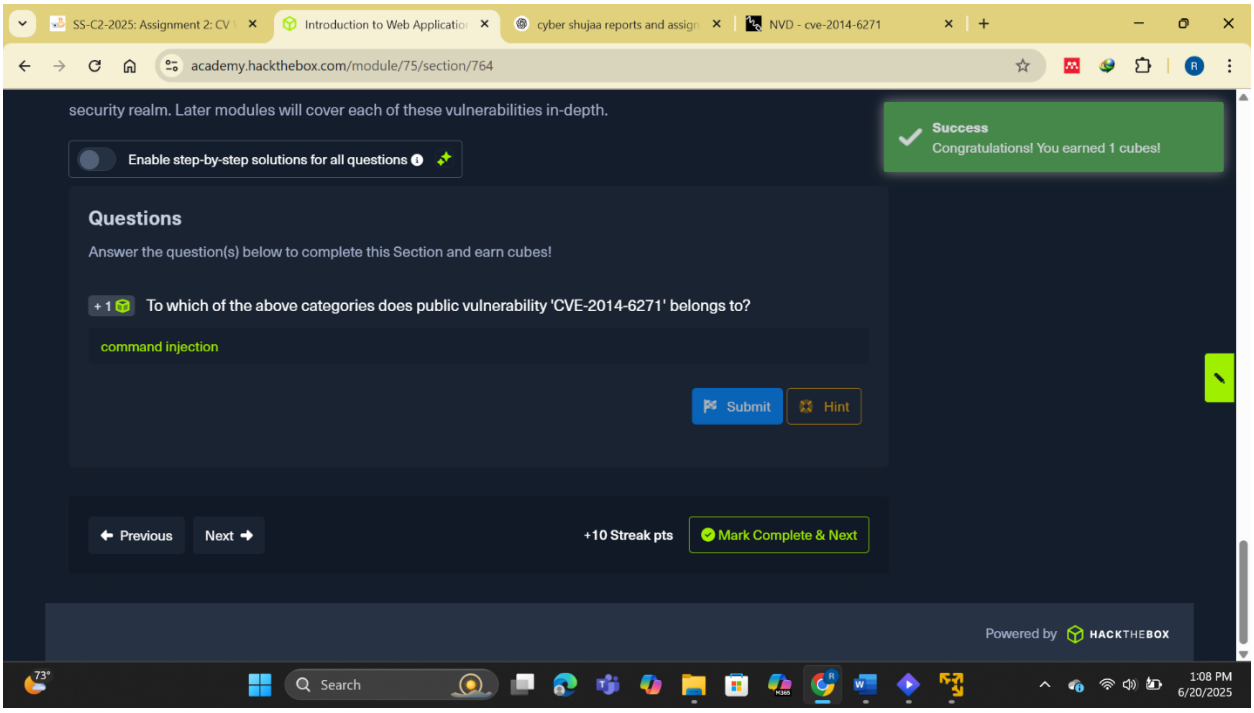

Fig 3.8 answers to questions on common vulnerabilities

Public Vulnerabilities

In this section, I explored the concept of **public vulnerabilities**, especially those that affect back-end components of web applications and can be exploited remotely. These vulnerabilities are often the result of coding errors made during development and, if left unpatched, can allow external attackers to gain full control of the server without requiring local access. This makes them a top priority in any external penetration test.

One of the first things I learned is the importance of **identifying the web application's version**. This can typically be found in files like version.php or in the source code of the application itself. Once identified, I can then begin searching for **publicly known exploits** associated with that version using databases like **Exploit DB**, **Rapid7**, or **Vulnerability Lab**. These resources often include **proof-of-concept (PoC)** exploits, CVE identifiers, and even full working scripts that can help in testing for vulnerabilities.

A vulnerability assigned a **CVE (Common Vulnerabilities and Exposures)** record typically includes detailed information such as the affected versions, the nature of the vulnerability, and its **CVSS (Common Vulnerability Scoring System)** score. The CVSS score helps me evaluate how severe a vulnerability is. For instance, using **CVSS v2.0**, a score of 7.0 to 10.0 is considered **High**, while **CVSS v3.0** categorizes anything from 9.0 to 10.0 as **Critical**. This scoring system allows organizations (and penetration testers like me) to **prioritize patching** and assess risk appropriately.

I also experimented with **CVSS calculators** provided by the National Vulnerability Database (NVD), which allow users to modify the **Temporal** and **Environmental** metrics to suit specific organizational contexts. Playing around with these calculators taught me how different factors like exploitability, scope, and impact can influence the final CVSS score.

While much of my focus is often on web applications themselves, I also learned that **vulnerabilities can exist in supporting components** such as the web server, the database, or even the operating system of the back-end server. A great example is the **Shellshock vulnerability (CVE-2014-6271)**, which affected **Apache web servers** and allowed attackers to remotely execute code through specially crafted HTTP requests. This highlights why even well-maintained servers need to be routinely updated.

Additionally, I explored how developers sometimes overlook security when using **third-party components** like plugins or themes. If a vulnerable plugin is deployed (even if the main application is secure), it can still serve as an entry point for attackers. This emphasizes the importance of treating every **external dependency** as a potential security risk.

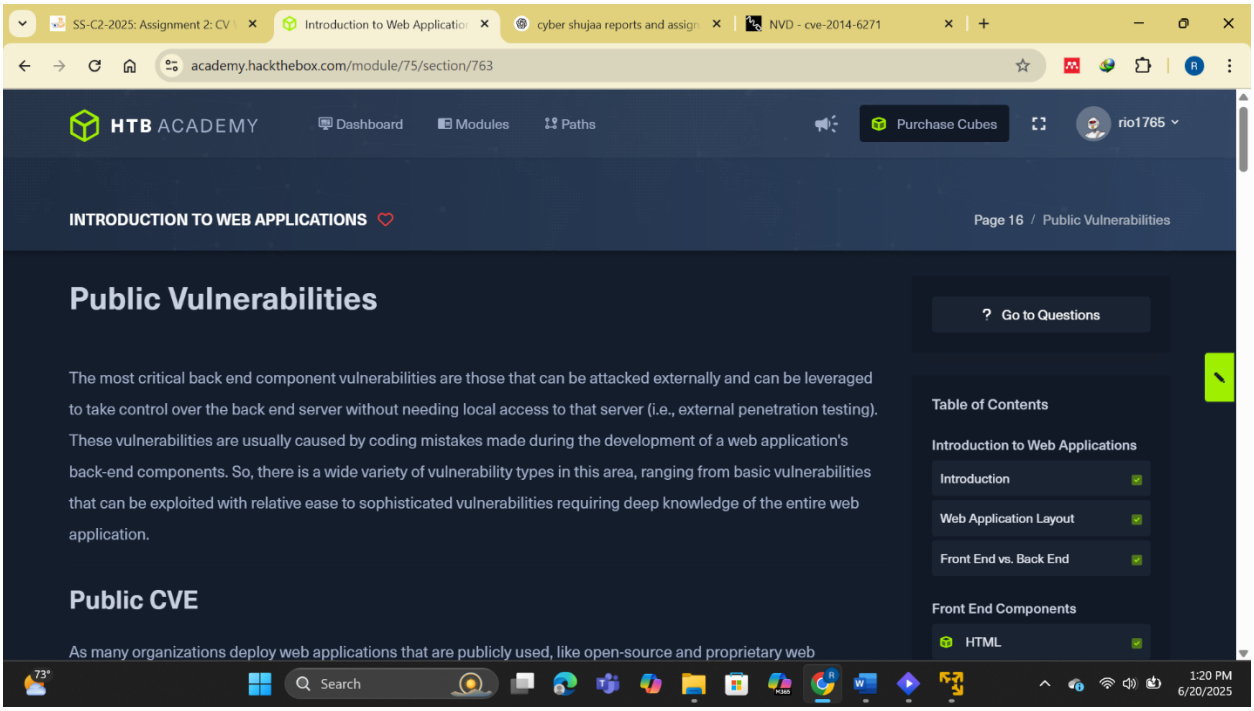

Fig 3.9 public vulnerabilities

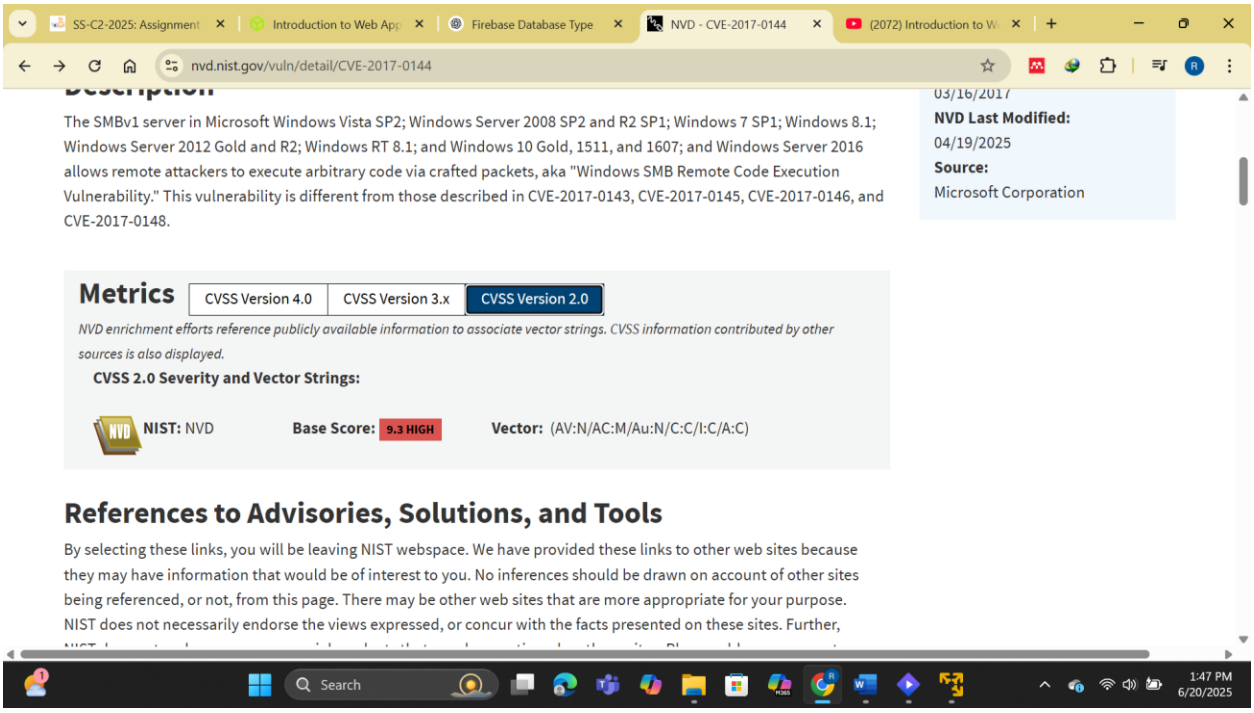

Fig 4.0 CVSS v2.0 score of the public vulnerability CVE-2017-0144

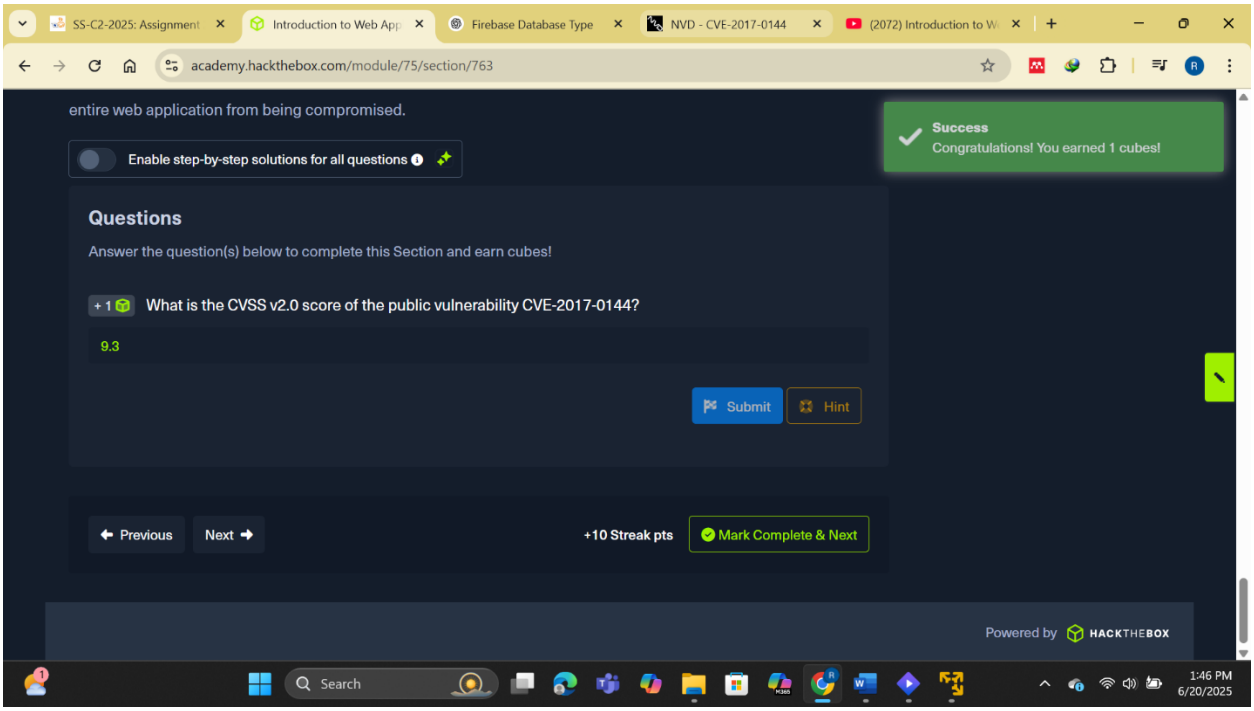

Fig 4.1 answer to question on public vulnerabilities

Next steps

Having reached the end of this module, I've gained a foundational understanding of **web application architecture**, its functionality, and the inherent **security risks** that come with it. From learning about front-end elements like HTML, CSS, and JavaScript to back-end systems involving web servers, databases, development frameworks, and APIs, I now appreciate the complexity and interconnectedness of modern web applications. I've also learned how web apps interact with users, store data, and communicate with other systems.

One of the key takeaways for me was recognizing how each layer—from the **presentation layer** in the browser to the **data layer** on the back-end server—can be a potential entry point for attackers if not secured properly. I now understand that even something as simple as failing to sanitize user input can lead to severe vulnerabilities like **HTML Injection, XSS, CSRF**, or **SQL Injection**. Equally, insecure implementations of **authentication**, **access control**, and **file upload** functions are common pitfalls that attackers regularly exploit.

To further build on my skills, I'm planning to go through modules like **Web Requests**, **JavaScript Deobfuscation**, and **Hacking WordPress**. These will help me understand common attack patterns, how web apps handle data under the hood, and how malicious scripts are often disguised and executed.

Conclusion

This module has offered a thorough overview of web applications, encompassing both their structural and functional aspects, as well as their associated security implications. By examining the roles of front-end technologies such as HTML, CSS, and JavaScript, alongside back-end components like web servers, databases, and development frameworks, I have developed a holistic understanding of how web applications operate. Furthermore, the analysis of prevalent vulnerabilities—including SQL injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and command injection—has underscored the importance of robust coding practices, proper input validation, and rigorous security testing. Moving forward, I intend to reinforce this knowledge through the practical development and assessment of web applications, with the objective of enhancing both functional proficiency and security awareness. Ultimately, mastering the intricacies of web application security is essential for safeguarding digital assets and ensuring the resilience of modern information systems.